PROJECT REPORT

# DESIGN AND IMPLEMENTATION OF A NETWORK MONITORING AND ANOMALY DETECTION SYSTEM ON THE PNETLAB PLATFORM

NETWORK MONITORING • INTRUSION DETECTION • ANOMALY ANALYSIS


PROJECT AUTHORS

**Tran Hoang Minh Vuong · Diep Manh Tuan**



Da Nang • 2026

# TABLE OF CONTENTS

**INTRODUCTION** 7
**CHAPTER 1. PROBLEM CONTEXT AND TECHNICAL FOUNDATIONS** 8
1.1 Network and security monitoring challenges 8
1.2 Collection and storage of operational metrics 9
1.3 Intrusion detection from network traffic 12
1.4 Anomaly detection using historical data 13
1.5 Application components and lab environment 15
**CHAPTER 2. REQUIREMENTS AND SYSTEM DESIGN** 18
2.1 Functional requirements and operational criteria 18
2.2 Network topology and information flow 20
2.3 Software architecture and alert life cycle 22
2.4 Detection and analytical mechanisms 28
2.5 Positioning and comparison with other solutions 29
**CHAPTER 3. IMPLEMENTATION OF NETMON SOC** 32
3.1 Preparing the emulation environment 32
3.2 Establishing the routed core and test services 35
3.3 Implementing the metrics and IDS pipelines 37
3.4 Backend, anomaly models, and alert processing 40
3.5 Operator interface and workflow 43
**CHAPTER 4. TESTING, RESULTS, AND ANALYSIS** 47
4.1 Test methodology and scope 47
4.2 Infrastructure-health monitoring tests 48
4.3 Intrusion-detection tests by network zone 51
4.4 Machine-learning and alert-life-cycle tests 59
4.5 Results assessment and measurement limits 62
**CHAPTER 5. OVERALL ASSESSMENT AND FUTURE WORK** 65
5.1 Key outcomes and project value 65
5.2 Technical and operational limitations 66
5.3 Development roadmap 67
**CONCLUSION** 69
**REFERENCES** 70

# PROJECT ABSTRACT AND KEY RESULTS

NetMon SOC is an experimental network and security monitoring environment developed to bring three complementary information sources into a single interface: operational metrics, intrusion-detection events, and anomaly alerts. Beyond reporting whether a device is reachable, the platform helps an operator investigate performance degradation, unusual traffic, and security events in their operational context. The purpose is to demonstrate how network monitoring and security analysis can be integrated within a reproducible laboratory architecture.

The PNETLab topology includes six monitored devices: a pfSense firewall, one Cisco IOSv router, two Layer 3 switches, and two Ubuntu servers. Prometheus collects measurements through SNMP Exporter and Node Exporter; Suricata running on pfSense writes EVE JSON events; and the Flask application combines these sources with PostgreSQL alert records, an Isolation Forest anomaly detector, Telegram notifications, and a Groq-powered AI assistant. The dashboards expose device health, security events, and investigation context from a common operational view.

Controlled validation was organized into three areas: infrastructure and resource monitoring, IDS event detection across network zones, and alert life-cycle behavior. Thirteen principal scenarios were compared with their observed outcomes, alongside additional security tests that exercised WAN, web, FTP, and DNS services. Results are interpreted using the recorded environment and evidence; an observed alert is not, by itself, a measurement of detection accuracy across an independent dataset.

The experiments show that metrics and Suricata events can travel through the collection, processing, visualization, and notification pipeline in the lab. The project demonstrates a practical integration pattern rather than a production-ready security operations center. Its main limitations concern the size of the emulated network, the duration and representativeness of training data, the lack of independent accuracy evaluation, and the absence of stress tests or production service-level objectives.

Keywords: NetMon SOC; PNETLab; Prometheus; SNMP; Suricata; Isolation Forest; Flask; PostgreSQL; Telegram; Groq.

# TERMS AND ABBREVIATIONS

| Term | Meaning in this report |
|---|---|
| SOC | Security Operations Center — platform for monitoring and analyzing security events. |
| NPM / NSM | Network Performance Monitoring / Network Security Monitoring — operational and security observation. |
| SNMP | Simple Network Management Protocol — collection of device status and interface counters. |
| OID / MIB | Object Identifier / Management Information Base — identifiers and data definitions used by SNMP. |
| TSDB / PromQL | Time-Series Database / Prometheus Query Language — storage and querying of time-indexed measurements. |
| IDS / IPS | Intrusion Detection / Intrusion Prevention System — detection and/or blocking of suspicious activity. |
| EVE JSON | Structured JSON event format emitted by Suricata. |
| API / REST | Application Programming Interface / Representational State Transfer. |
| ML / LLM | Machine Learning / Large Language Model — learned analytics and language generation. |
| DMZ / VLAN | Demilitarized Zone / Virtual Local Area Network — separated service network / logical LAN. |
| OSPF | Open Shortest Path First — an interior gateway routing protocol. |
| MTTR | Mean Time to Repair — average duration needed to restore a failed service. |

# LIST OF FIGURES

Figure 2.1. Overall network topology of the NetMon SOC laboratory 20
Figure 2.2. Monitoring-data flows across NetMon SOC components 22
Figure 2.3. Layered software architecture of NetMon SOC 24
Figure 2.4. Project source-code directory structure 26
Figure 2.5. Alert life cycle and state transitions 28
Figure 3.1. PNETLab network topology used in the experiment 34
Figure 3.2. Real-time device-monitoring dashboard 44
Figure 3.3. Alert management interface 44
Figure 3.4. SOC dashboard for Suricata IDS events 45
Figure 3.5. AI Agent interface with live network context 46
Figure 4.1. Telegram alerts for pfSense outage and automatic recovery 49
Figure 4.2. Main NetMon WAN Attack Console interface 53
Figure 4.3. Web Attack Generator interface 55
Figure 4.4. FTP Attack Menu interface 56
Figure 4.5. DNS Attack Menu interface 57
Figure 4.6. AI anomaly alerts for Server02 CPU and memory utilization 61

# LIST OF TABLES

Table 2.1. IP address allocation by network segment 20
Table 2.2. Schema of the alerts table for operational alerts 24
Table 2.3. Schema of the soc_events table for Suricata IDS events 25
Table 2.4. Comparison of NetMon SOC with reference solutions 30
Table 3.1. Software versions used in the system 32
Table 3.2. Monitored devices and their functional roles 33
Table 4.1. Summary of the primary validation scenarios 47
Table 4.2. Results of server-resource threshold testing 50
Table 4.3. Suricata IDS validation results after additional scenarios 58
Table 4.4. Results of alert life-cycle validation 62
Table 4.5. Consolidated results for the primary test scenarios 62
Table 4.6. Detection delays observed by alert category 63

# INTRODUCTION

Device-monitoring tools typically answer whether infrastructure is operating, whereas security-monitoring tools look for suspicious traffic and access patterns. When these information sources remain separate, correlating a performance alert with a security event becomes difficult. NetMon SOC was built to bring operational metrics, Suricata event records, and machine-learning anomaly notifications into a common investigation workflow, helping an operator narrow down likely causes before taking remedial action.

The project team created a compact enterprise topology in PNETLab, deployed SNMP and Node Exporter collection, integrated an intrusion-detection system, designed alerting rules, and executed controlled experiments. Each scenario followed an evidence-based sequence: establish the initial state, introduce a controlled stimulus, collect source measurements, compare the displayed status, and verify notifications. The findings refer only to this laboratory environment; the project does not establish scalability at enterprise size, production-grade security, or service-level guarantees.

# CHAPTER 1. PROBLEM CONTEXT AND TECHNICAL FOUNDATIONS

## 1.1 Network and security monitoring challenges

### *1.1.1 The role of proactive monitoring*

Network monitoring continuously collects and evaluates infrastructure status to identify connectivity loss, degraded performance, or behavior that warrants investigation. The monitored estate can include routers, switches, firewalls, servers, and application services. A measurement becomes operationally useful when its source, timestamp, and device context are known.

The project combines two complementary viewpoints. Network Performance Monitoring (NPM) focuses on availability, traffic, interfaces, and resource utilization. Network Security Monitoring (NSM) records suspicious activities and protocol events found in network traffic and logs.

In day-to-day operations, proactive monitoring should support early detection, trend analysis, and faster fault isolation. Measurements such as interface state, latency, CPU load, and packet errors become actionable only when associated with appropriate thresholds, historical context, and a defined response. Security logs enrich that context by indicating which patterns were observed at particular times.

Modern monitoring is difficult not only because of the number of assets, but also because sources differ in protocol, sampling frequency, timestamp format, data structure, and reliability. SNMP counters, operating-system metrics, and application-layer security events describe different phenomena. Before presenting them in one dashboard, a system must establish consistent device identities, time references, network-zone labels, and storage conventions, otherwise apparently related graphs and alerts may describe unrelated events.

False positives and false negatives create opposing operational risks. Overly sensitive thresholds can overwhelm responders with repetitive notifications, while weak thresholds may allow meaningful incidents to go unnoticed. Debouncing, controlled

reminders, and cross-checking multiple sources can reduce alert noise, but they do not prove a detector's statistical quality.

### *1.1.2 Requirements addressed by NetMon SOC*

The monitoring problem is expressed through three questions: which device or service is deteriorating, when did the abnormal condition begin, and which evidence can help isolate the fault? Device availability, service availability, and security activity must be distinguished. For example, an ICMP-responsive server may still have an unavailable HTTP endpoint, while a sudden traffic increase may be a legitimate workload rather than a denial-of-service attack.

To support investigation, NetMon SOC links device names and network zones with resource metrics, Suricata events, and event timestamps. If elevated CPU usage coincides with an IDS notification, the operator can examine both signals in a common view and formulate a hypothesis. The causal relationship still requires verification through process-level observations, packet paths, and source logs.

Three principal outputs guide the design: a dashboard for current health and activity, historical data for follow-up queries, and notifications directed to the responsible operator. Automated IP blocking, endpoint isolation, configuration rollback, and autonomous incident response were not implemented or validated as part of the project.

## 1.2 Collection and storage of operational metrics

### *1.2.1 SNMP and network-device information*

The Simple Network Management Protocol (SNMP), specified in RFC 1157 for SNMPv1, RFC 1901 for SNMPv2c, and the RFC 3411–3418 family for SNMPv3, is commonly used to manage and observe network devices. SNMP follows a manager-agent architecture: the monitoring manager queries an agent, typically over UDP port 161; the agent retrieves values defined by its Management Information Base (MIB) and returns them to the requester.

Managed objects are addressed through hierarchical Object Identifiers (OIDs). For example, OID 1.3.6.1.2.1.2.2.1.8, known as ifOperStatus, represents an interface's

operational state: 1 indicates up, 2 down, and 3 testing. In this project, the IF-MIB is used to observe interface availability, administrative state, traffic counters, nominal speed, and operator-defined descriptions.

ifOperStatus: operational state of a network interface.

ifAdminStatus: administrative state configured for the interface.

ifHCInOctets / ifHCOutOctets: cumulative 64-bit received and transmitted byte counters.

ifHighSpeed: interface speed expressed in megabits per second.

ifAlias: descriptive interface label configured by the administrator.

SNMPv2c was selected for compatibility with the pfSense and Cisco IOSv nodes in the isolated laboratory, and because it supports high-capacity 64-bit interface counters. Its community strings are not encrypted, however. A deployment outside this controlled environment should favor SNMPv3, restrict management access to approved sources, and protect monitoring credentials and the collected operational information.

### *1.2.2 Node Exporter and Linux host metrics*

Node Exporter is part of the Prometheus ecosystem. It runs as a Linux daemon and exposes operating-system measurements at an HTTP /metrics endpoint, commonly on TCP port 9100. Prometheus scrapes this endpoint periodically and stores the resulting labeled time series.

node_cpu_seconds_total{mode="idle"}: cumulative idle CPU time; a typical utilization estimate is CPU% = 100 − avg(irate(idle[5m])) × 100.

node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes: available and total memory used to derive memory-utilization percentage.

node_filesystem_size_bytes / node_filesystem_avail_bytes: filesystem capacity and available storage space.

node_network_receive_bytes_total / node_network_transmit_bytes_total: received and transmitted network traffic by interface.

Node Exporter complements the SNMP-based network inventory with operating-system measurements such as CPU load, memory pressure, and storage utilization. The two collection mechanisms should not be ranked as universally more or less accurate: they expose different observations, for different device classes, under different sampling and access conditions.

### *1.2.3 Prometheus, time series, and queries*

Prometheus is an open-source monitoring system and time-series database that originated at SoundCloud and is now part of the Cloud Native Computing Foundation ecosystem. Its collection model is primarily pull-based: the Prometheus server requests metrics from configured targets at a specified interval.

Every Prometheus time series has a metric name and a set of key-value labels. An interface-state observation might be represented as ifOperStatus{job="snmp-pfsense", instance="172.24.0.115", ifIndex="1"}. PromQL supports scalar and vector operations, aggregation, filtering, and range functions, allowing the application to calculate rates from cumulative counters or compare resource utilization against thresholds over time.

NetMon SOC configures a 15-second scrape interval and six jobs covering its six monitored nodes. Four SNMP jobs are named snmp-pfsense, snmp-core-router, snmp-swl3a, and snmp-swl3b; two Node Exporter jobs are named node-server01 and node-server02. Prometheus stores their metrics locally and exposes the HTTP endpoints /api/v1/query and /api/v1/query_range for instantaneous and time-window queries, which the Flask application uses to populate its dashboards.

A 15-second scrape cycle offers near-real-time observations, not continuous capture. An event that begins and ends between samples may not appear as a distinct data point. Suricata's event logs and active service-health checks therefore provide complementary evidence, while the interface should communicate the timestamp of its most recent successful update.

## 1.3 Intrusion detection from network traffic

### *1.3.1 IDS concepts and Suricata processing*

An Intrusion Detection System (IDS) monitors network traffic or host activities to identify suspected attacks, unauthorized activity, or policy violations. Network IDS sensors examine traffic traversing an observation point, whereas Host IDS tools inspect events on a particular machine. Detection may rely on known attack signatures or deviations from a learned or defined baseline. NetMon SOC combines signature-based Suricata observations with a separate metrics-oriented anomaly-detection module.

An IDS records evidence and raises alerts; it does not necessarily block the traffic that triggered a rule. An Intrusion Prevention System (IPS) may actively drop traffic when deployed inline. Suricata can support both deployment modes, but the project uses it in the monitoring role demonstrated in the lab.

Suricata is a multithreaded open-source NIDS/NIPS engine maintained by the Open Information Security Foundation. It can analyze several protocols in parallel and match traffic against a set of Suricata-compatible detection rules.

Its processing sequence can be summarized as capture, decode, detect, and log. Capture collects packets through supported capture mechanisms; decoding interprets Ethernet, IP, TCP/UDP, HTTP, DNS, and TLS information; detection evaluates enabled signatures; and logging emits structured event records. A central output is EVE JSON, with one JSON object per line, representing alerts or supported protocol events. This format is useful for ingestion because it can be parsed and filtered without relying on free-form syslog messages.

In the laboratory, Suricata runs directly on pfSense and monitors five interfaces: WAN (vtnet125952), Web-DMZ (vtnet33887), DNS-FTP (vtnet450526), LAN (vtnet210674), and ZeroTier (vtnet062106). Each monitored interface writes to its own EVE JSON file under /var/log/suricata/suricata_<interface>/eve.json.

### *1.3.2 EVE JSON and SOC analysis records*

In the EVE JSON format, each line is a standalone JSON object describing an event. Typical alert data include timestamp; event_type such as alert, dns, http, or tls; src_ip,

src_port, dest_ip, dest_port, and proto; plus the nested alert.signature, alert.category, alert.severity, alert.gid, and alert.signature_id fields. These attributes support chronological reconstruction, source-destination analysis, rule identification, severity categorization, and filtering by protocol or zone.

NetMon SOC maps Suricata priorities to application severities: priority 1 becomes Critical, priority 2 becomes Warning, and priorities 3 and 4 become Info. Longer Suricata categories are also normalized for display: for example, "Web Application Attack" becomes "Web Attack," while "Detection of a Network Scan" becomes "Port Scan." The original signature identifier remains important because it provides the traceable connection between a dashboard event and its triggering rule.

An EVE JSON event is not necessarily a security alert: some records describe ordinary DNS or HTTP transactions, and the nested alert object is present only when a rule matches. The ingestion process therefore normalizes timestamps, zones, interfaces, source and destination addresses, and signature metadata before storage and display. Preserving those fields enables an operator to check whether multiple dashboard entries refer to the same origin, destination, or detection rule rather than treating every row as an unrelated incident.

## 1.4 Anomaly detection using historical data

### *1.4.1 Principles of Isolation Forest*

Anomaly detection identifies observations that differ substantially from the rest of a dataset. In network operations, such observations may include a sudden traffic spike, unexpected resource consumption, or unusual changes in activity over time.

Isolation Forest, introduced by Liu, Ting, and Zhou in 2008, is an unsupervised outlier-detection algorithm based on randomized partition trees. The underlying idea is that unusual observations tend to be isolated by fewer random splits than common observations. The model builds multiple isolation trees, selecting features and split values at random; the typical isolation path length across those trees is then used to estimate how exceptional a point is within the training distribution.

Points that fall within dense regions generally require deeper partitioning, while isolated points tend to have shorter paths. The anomaly score can be expressed as s(x, n) = 2^(−E(h(x)) / c(n)), where E(h(x)) is the average path length of x and c(n) normalizes the expected path length for a sample of size n.

A score approaching 1 suggests a strong outlier, whereas a score nearer 0 represents a more typical observation under the fitted model. The result is relative to the chosen features and training period.

### *1.4.2 Feature selection and training data*

In NetMon SOC, the Isolation Forest module considers seven operational metrics grouped by device class.

Network appliances (firewall, router, switch): traffic_in_bps, traffic_out_bps, in_errors, and out_errors, representing inbound and outbound rates and interface error measurements.

Linux servers: cpu_percent, mem_used_pct, and disk_used_pct, representing processor, memory, and storage utilization.

Each modeled metric is stored in its own .pkl artifact together with a StandardScaler so that prediction uses the same feature transformation as training. The configured training window is seven days (168 hours) of Prometheus history sampled every five minutes, or about 2,016 samples per metric when data are complete. The model uses n_estimators=200 and random_state=42 for repeatable training, while contamination=0.05 expresses an assumed five-percent outlier proportion during fitting.

The seven-day historical baseline is appropriate for a reproducible laboratory demonstration, but it is not a representative production benchmark by itself. In particular, contamination=0.05 is an algorithm configuration parameter rather than a measured false-positive rate.

## 1.5 Application components and lab environment

### *1.5.1 Flask, REST APIs, and PostgreSQL*

Flask is a lightweight Python web framework that allows applications to organize presentation and business logic without enforcing one architecture. NetMon SOC uses Flask in two roles: server-side rendering of HTML pages through Jinja2 templates, and a REST API backend that supplies JSON data to the browser's JavaScript components. This design keeps the operator interface and the programmatic retrieval endpoints within the same application while separating them into purpose-specific modules.

The REST endpoints are grouped through Flask Blueprints: dashboard for device monitoring, alerts for alert management, soc for Suricata event analysis, and ai_agent for LLM-assisted investigation. GET requests retrieve status or historical information, while POST requests initiate supported actions.

PostgreSQL serves as the relational database for structured alerts and operational events. It supports filtering and ordering on several fields and can use composite indexes for frequently executed query patterns. SQLAlchemy ORM provides a data-access abstraction that limits the coupling between application logic and individual SQL statements.

Three principal tables are documented: alerts for the operational alert life cycle, soc_events for normalized Suricata records, and service_checks for endpoint health-check observations. The schema and indexes are organized around common queries such as recent events, active alerts, severity filters, and device-specific histories.

The application uses a SQLAlchemy Engine connection pool configured with DB_POOL_SIZE=10, DB_MAX_OVERFLOW=20, and DB_POOL_TIMEOUT=30 seconds. These parameters support concurrent background tasks without opening a new database connection for every operation.

### *1.5.2 Telegram notifications and Groq LLM assistance*

The notification subsystem is structured around an abstract NotificationChannel interface. Concrete channels, such as TelegramChannel, implement send(recipient_config, message); an EmailChannel placeholder illustrates how other

transports could be added. NotificationRouter selects recipients based on role, alert category, and severity, then dispatches messages through the appropriate registered channel. This reduces coupling between alert-generation logic and the messaging transport.

Telegram Bot API is the implemented notification channel, using HTML-formatted requests to the endpoint https://api.telegram.org/bot{token}/sendMessage. Each configured team member has a chat_id stored outside the source code in environment variables.

The alert-routing rules distinguish a leader from a regular member. A leader may receive critical, warning, AI anomaly, Suricata, resolved, daily summary, and service-down notifications. Members receive alerts associated with their assigned devices and selected high-priority categories, including critical, warning, service-down, and Suricata. This design seeks to keep important information visible while reducing irrelevant notifications and operator fatigue.

The AI Agent follows a context-augmented generation pattern. Before answering an operator's question, the application retrieves current Prometheus measurements and recent PostgreSQL alert records, then places this information in the LLM request context through the Groq API. The prompt constrains answers to observed data, encourages practical CLI checks, and supports Vietnamese or English according to the user's language. Up to 16 conversation turns are retained in memory for short-term context; generated recommendations still require human validation.

### *1.5.3 PNETLab, ZeroTier, and pfSense*

PNETLab provides an emulation environment in which pfSense, Cisco IOSv routers and switches, and Linux servers can run together on a virtualized host. The team uses it to control connectivity, manipulate interface and device states, and repeat security or monitoring tests without affecting production infrastructure.

ZeroTier is configured on pfSense to provide an overlay management connection from the operator's host to the PNETLab network. Through pfSense's ZeroTier address, 172.24.0.115, an external administration workstation can reach the lab's 10.20.0.0/16 address space according to routing and firewall policy. Suricata also monitors the

ZeroTier interface, allowing the project to observe events associated with management access as a separate zone.

pfSense is a FreeBSD-based firewall and gateway platform that supports NAT, access-control rules, and additional packages, including IDS software. Installing Suricata on the pfSense node places monitoring close to a common routing and security boundary, where multiple segmented networks exchange traffic.

In NetMon SOC, pfSense simultaneously acts as the gateway/NAT device, an SNMP agent, the Suricata host, and the ZeroTier endpoint. Consolidation reduces the number of virtual machines but also introduces a shared dependency: a pfSense fault can affect connectivity, management access, and security observation at the same time.

# CHAPTER 2. REQUIREMENTS AND SYSTEM DESIGN

## 2.1 Functional requirements and operational criteria

### *2.1.1 Monitoring, detection, and alerting requirements*

The system is designed to monitor six target devices through four SNMP jobs and two Node Exporter jobs, displaying connectivity, interface state, traffic, and server resource use. The configured metrics scrape interval is 15 seconds. Additional endpoint checks for web, TCP, and DNS services are specified at a 30-second interval to identify application failures that cannot be inferred from device availability alone.

Device Monitoring: The application collects network-device metrics using SNMP and Linux-server metrics using Node Exporter every 15 seconds. For each monitored asset it reports online/offline state, interface up/down status, interface traffic in both directions, error rate, and uptime. Linux hosts also expose CPU, RAM, and disk utilization. The dashboard presents near-real-time observations together with historical charts covering the preceding 60 minutes, enabling operators to relate current status to recent trends.

Service Health Checks: The design checks selected endpoints at a nominal 30-second interval using HTTP/HTTPS, TCP, and DNS. An HTTP check records response status and latency, a TCP check attempts to establish a connection to a configured port, and a DNS check tests name resolution.

IDS Integration: A background worker retrieves Suricata EVE JSON data from pfSense over SSH every 10 seconds, parses and deduplicates the incoming records, and stores security alerts in PostgreSQL. The SOC interface is intended to show event counts by severity, distributions across protocols and network zones, frequent source addresses, an hourly event timeline, and searchable event details.

Alerting rules cover CPU above 85%, RAM above 90%, disk utilization above 85%, traffic over 800 Mbps, errors above 10 per second, and disabled or failed interfaces. To reduce noise, selected conditions must persist across three successive failed observations before an alert is confirmed. Recovery also passes through a waiting period before the

alert is automatically closed. These figures describe the documented configuration and should be cross-checked against the settings loaded by the running application.

AI Agent: The user interface provides a chat panel where an operator can ask about network status in natural Vietnamese or English. Answers should be grounded in the actual metrics and alert records supplied by the application rather than fabricated network facts. Where appropriate, the assistant suggests specific diagnostic CLI commands.

### *2.1.2 Performance, security, and maintainability requirements*

The interface design targets an approximately two-second response time while collecting metrics on a 15-second cycle. These values are acceptance criteria rather than universal performance guarantees.

For stability, the application uses a 12-second cache TTL to reduce redundant Prometheus requests and a PostgreSQL connection pool configured with 10 baseline connections plus up to 20 overflow connections. Exception handling in each background worker is intended to contain isolated task failures.

Maintainability: Operational settings are centralized in config.py, so new devices, team members, or thresholds can be introduced through configuration rather than scattering changes across handlers.

Security: Telegram tokens, Groq API keys, and SSH connection details are loaded from environment variables through a .env file rather than embedded in committed source code. The connection to pfSense uses SSH key-based authentication.

Extensibility: A new monitored device can be registered by adding an entry to the DEVICES collection in config.py. Additional notification transports, such as email or Slack, can be implemented by extending NotificationChannel and registering the new handler.

Moving credentials out of source code is necessary, but not sufficient, to secure the service. The tested version does not provide a complete login and role-based authorization system for the web interface. A deployment beyond the lab should add HTTPS, user authentication, access controls for sensitive pages and APIs, activity

logging, secret rotation, and an appropriate hardening process before exposing any functionality to untrusted networks.

## 2.2 Network topology and information flow

### *2.2.1 Components and connectivity in the laboratory*

The network follows a segmented small-enterprise topology in which pfSense acts as the central gateway and policy boundary. A core router connects the routed backbone, two Layer 3 switches provide internal distribution, and Linux servers host the test applications in dedicated demilitarized zones (DMZs).

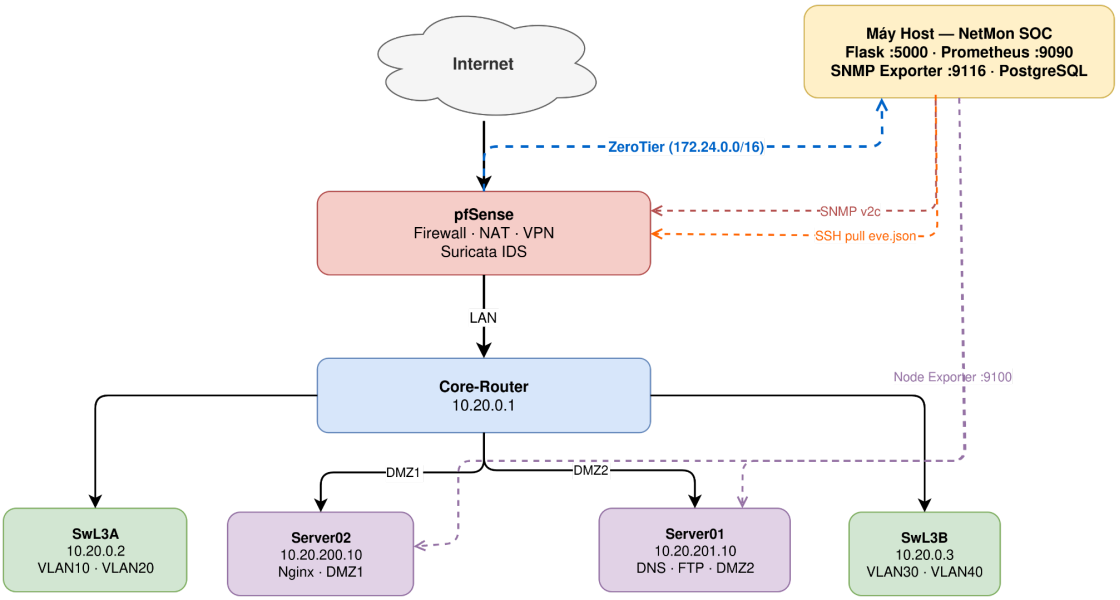



*Figure 2.1. Overall network topology of the NetMon SOC laboratory*

### *2.2.2 Network zones and IP address plan*

The laboratory uses 10.20.0.0/16 as its internal address space. It is divided into functional subnets for management, departmental VLANs, and the application DMZs.

*Table 2.1. IP address allocation by network segment*

| Network segment | Subnet | Gateway | Device / Description |
|---|---|---|---|
| Management | 10.20.0.0/24 | — | Core-Router (10.20.0.1), SwL3A (10.20.0.2), SwL3B (10.20.0.3) |
| VLAN10 — IT | 10.20.10.0/24 | 10.20.10.1 | IT department users |

| Network segment | Subnet | Gateway | Device / Description |
|---|---|---|---|
| VLAN20 — Sales | 10.20.20.0/24 | 10.20.20.1 | Sales department users |
| VLAN30 — HR | 10.20.30.0/24 | 10.20.30.1 | Human Resources users |
| VLAN40 — Engineering | 10.20.40.0/24 | 10.20.40.1 | Engineering department users |
| DMZ1 — Web | 10.20.200.0/24 | 10.20.200.1 | Server02 — Nginx Web Server (10.20.200.10) |
| DMZ2 — DNS/FTP | 10.20.201.0/24 | 10.20.201.1 | Server01 — BIND9 DNS and vsftpd FTP (10.20.201.10) |
| ZeroTier Overlay | 172.24.0.0/16 | — | pfSense ZeroTier (172.24.0.115) and administration workstation |

### *2.2.3 Collection paths from devices to the application*

Three collection paths are distinguished. Router, switch, and firewall metrics pass through SNMP Exporter; Linux host metrics come from Node Exporter; and Suricata security events are read as EVE JSON. The two metrics paths enter Prometheus, while normalized IDS alerts are stored in PostgreSQL.

Path 1 — SNMP Metrics: SNMP Exporter, listening on TCP port 9116, is used with Prometheus to query the four network devices: pfSense, Core-Router, SwL3A, and SwL3B. The configured collection cycle is 15 seconds. Prometheus stores the resulting time-series samples, and NetMon SOC queries its HTTP API for current values, historical traffic rates, and interface states to display in the operator dashboard.

Path 2 — Node Exporter Metrics: Prometheus directly scrapes the two Node Exporter endpoints on Server01 and Server02 at port 9100 every 15 seconds. Linux CPU, memory, filesystem, and network information is stored as labeled samples in the time-series database.

Path 3 — Suricata IDS Events: A NetMon SOC worker establishes an SSH connection to pfSense at 172.24.0.115 on a 10-second schedule. It reads the latest N lines, with N=500, from per-interface eve.json logs, parses each JSON object, removes

duplicates, and inserts normalized security records into the PostgreSQL soc_events table.

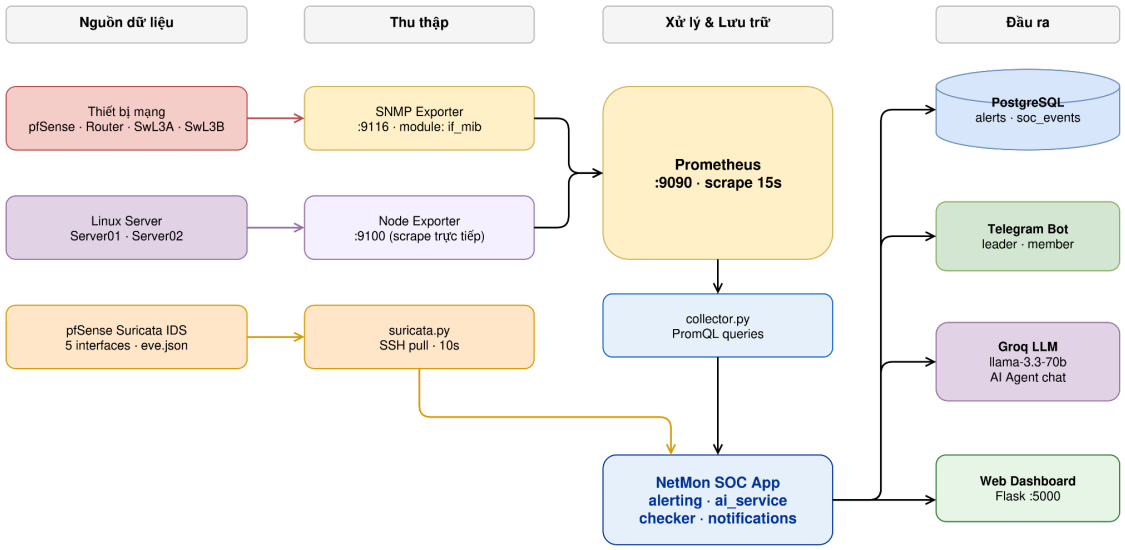



***Figure 2.2. Monitoring-data flows across NetMon SOC components***

The complete pipeline is checked from source to consumer: devices expose metrics, Prometheus stores samples, NetMon SOC queries an API, and the dashboard renders the results. In parallel, Suricata produces EVE JSON that the application retrieves over a management channel and persists as security events.

## 2.3 Software architecture and alert life cycle

### *2.3.1 Responsibilities of the application layers*

NetMon SOC adopts a four-layer application architecture to separate user interaction, request handling, operational logic, and persistence.

Presentation Layer: Jinja2 templates, HTML, CSS, JavaScript, and assets in templates/ and static/ render the dashboard and expose operator controls. Browser-side JavaScript requests data from REST endpoints and updates status widgets or charts without a complete page reload.

Application or Route Layer: Flask Blueprints in routes/ include dashboard.py, alerts.py, soc.py, and ai_agent.py. They accept HTTP requests, validate parameters, invoke the appropriate service, and return JSON or an HTML template.

Service or Business Logic Layer: Modules in services/ include collector.py for Prometheus retrieval, alerting.py for alert rules, suricata.py for IDS ingestion, checker.py for endpoint tests, notifications.py for messaging, ai_service.py for AI assistance and anomaly detection, and timezone.py for timestamp handling. This layer coordinates background workers and encapsulates the decisions that translate raw observations into operational information.

Data Access Layer: database/models.py defines ORM models; database/repository.py centralizes queries; and database/__init__.py prepares the SQLAlchemy Engine and session factory. The Repository Pattern isolates storage operations from the REST and service layers.

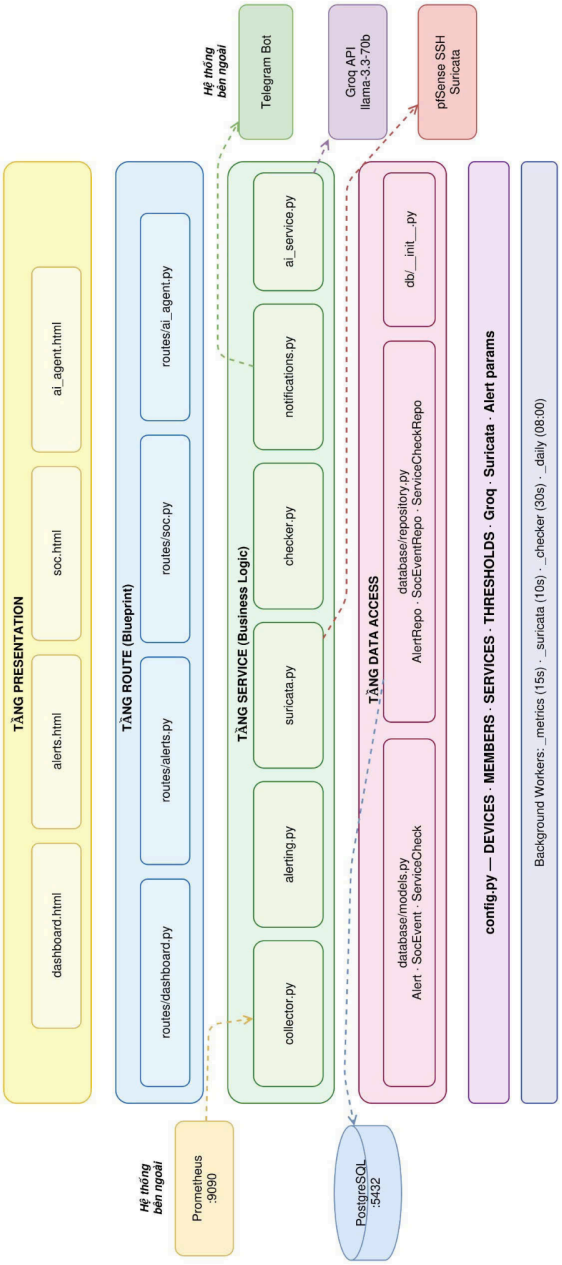



*Figure 2.3. Layered software architecture of NetMon SOC*

### 2.3.2 Data model and source-code organization

NetMon SOC uses three primary relational tables to support the operational requirements while limiting unnecessary duplication.

*Table 2.2. Schema of the alerts table for operational alerts*

| Column | Data type | Constraint | Description |
|---|---|---|---|
| id | String(36) | PRIMARY KEY | UUID v4 uniquely identifying an alert |
| created_at | DateTime | NOT NULL | Alert creation time |

| Column | Data type | Constraint | Description |
|---|---|---|---|
| device | String(64) | NOT NULL | Device name (pfSense, Core-Router, etc.) |
| severity | String(16) | NOT NULL | Severity: critical, warning, info |
| category | String(32) | NOT NULL | Alert category: cpu, memory, interface, suricata, ai-anomaly, etc. |
| message | Text | NOT NULL | Short alert description |
| details | Text | — | Additional supporting information |
| sub_key | String(128) | — | Condition key for debouncing and automatic resolution |
| resolved | Boolean | NOT NULL | Whether the alert has been resolved |
| resolved_by | String(64) | — | Resolving username or "system" |
| resolved_at | DateTime | — | Resolution timestamp |
| resolved_note | Text | — | Resolution note |
| resolve_mode | String(16) | — | Resolution mechanism: human, auto, manual |

*Table 2.3. Schema of the soc_events table for Suricata IDS events*

| Column | Data type | Description |
|---|---|---|
| id | BigInteger | Auto-incrementing primary key |
| occurred_at | DateTime | Original event occurrence time |
| src_ip / src_port | String(45) / Integer | Source address and port |
| dest_ip / dest_port | String(45) / Integer | Destination address and port |
| proto | String(8) | Protocol (TCP, UDP, ICMP) |
| severity | String(16) | Severity: critical, warning, info |
| priority | SmallInteger | Suricata priority (1–4) |
| sig_id | Integer | ID of the detection signature |
| signature | Text | Name of the triggering signature |
| category | String(64) | Normalized event category |
| http_url / http_method / http_status | Text/String/SmallInt | HTTP request information, where available |

| Column | Data type | Description |
|---|---|---|
| interface_name | String(32) | Monitored interface (WAN, Web-DMZ, etc.) |
| zone | String(32) | Network zone (wan, dmz-web, lan, etc.) |

The Repository Pattern is applied consistently through the data layer. Three repositories are defined: AlertRepository, SocEventRepository, and ServiceCheckRepository. Each owns queries related to its entity type.

Database access is organized around the get_db() context manager, which supplies a SQLAlchemy session and handles commit or rollback as appropriate. Repository methods return ordinary Python dictionaries and lists instead of exposing ORM objects beyond an active session.

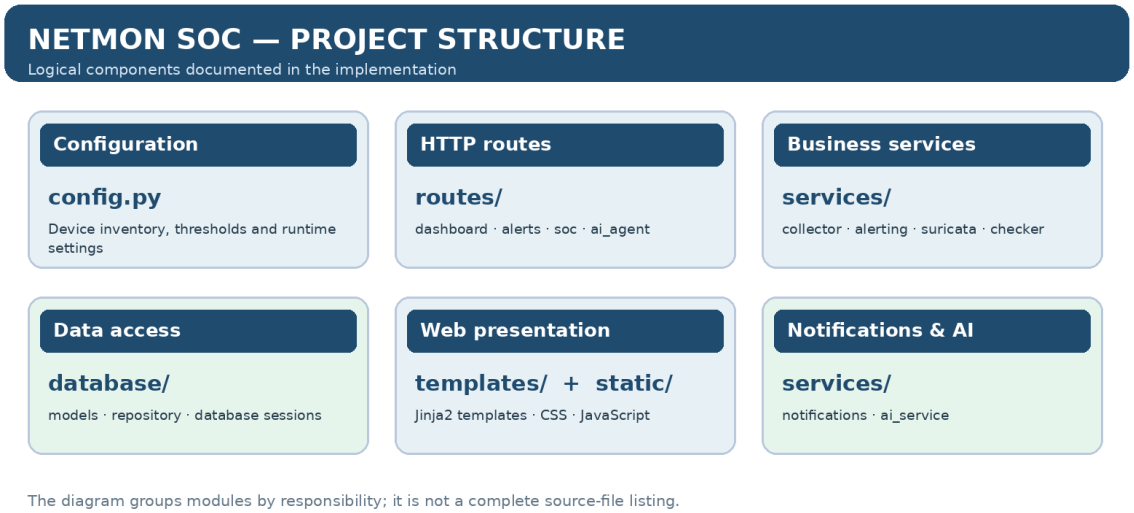



*Figure 2.4. Project source-code directory structure*

Each data store serves a different purpose. Prometheus retains metric histories by time and label; the alerts table records the evolving status of a single operational condition; and soc_events retains IDS observations together with timestamps and signatures. This separation allows an alert to be resolved without overwriting the underlying evidence and supports different retention and query policies for sampled metrics and event records.

Identifiers such as device, sub_key, severity, status, and created_at support incident filtering by source and time. When several workers observe or update the same condition, database operations must prevent duplicate active alerts. Centralized

repository queries and transactions provide one place to enforce consistency, rather than scattering these decisions across views, API handlers, and scheduled tasks.

### *2.3.3 Rules for creating, reminding, and resolving alerts*

The documented alert life cycle has five conceptual states: Counting, Confirmed, Active, Resolving, and Resolved. It is designed to balance responsiveness with notification noise.

Counting: When a condition is detected, such as CPU utilization above 85%, the application increments fail_count for the corresponding sub_key. An alert is confirmed only after PING_FAIL_THRESHOLD=3 consecutive failed observations, according to the rule's implementation.

Confirmed: Once the failure threshold is reached, the application creates an alert record in the database and sends a Telegram notification to the selected recipients.

Active: If the monitored condition continues, the system sends a reminder every REMINDER_INTERVAL_SECONDS=600 seconds, equivalent to ten minutes.

Resolving: When the triggering condition stops, fail_count is reset and a grace deadline is established using AUTO_RESOLVE_GRACE=300 seconds, or five minutes. During this period the system waits for the healthy condition to remain stable.

Resolved: After the grace period expires, the alert is marked resolved with resolve_mode="auto" and the leader receives a recovery notification. REOPEN_GRACE_SECONDS=120 provides an additional cooldown before the same sub_key can open a new incident.

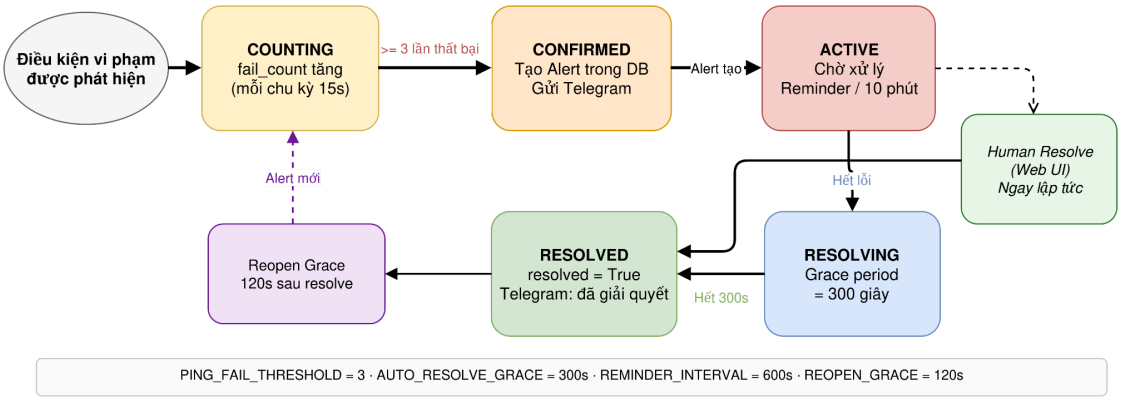



*Figure 2.5. Alert life cycle and state transitions*

In operational terms, an alert is not merely either failed or healthy. The system first accumulates consecutive violations, then confirms and maintains an incident identity while a condition persists. Controlled reminders communicate ongoing impact without creating an unlimited number of duplicate records. Recovery must remain stable through the 300-second grace period, and the subsequent reopen delay limits oscillation when a metric is unstable around its threshold.

## 2.4 Detection and analytical mechanisms

### *2.4.1 Correlating operational metrics with IDS events*

Prometheus measurements are indexed by device, metric labels, and time, whereas Suricata events include source and destination addresses, a signature, and severity. Displaying these sources together can help an operator determine whether a CPU spike occurred around the same time as unusual connection attempts. Such proximity is a useful investigative cue, not proof that the connections caused the resource change; additional host and packet-level evidence is required.

For threshold-based metrics, the system combines a configured limit with a count of consecutive violations to suppress short spikes. For Isolation Forest, an outlier score acts as a separate signal for investigation. A statistical deviation does not automatically establish that an intrusion occurred, even when another monitoring component generated a warning in the same period.

The threshold detector evaluates CPU, RAM, disk, and interface conditions based on operator-configured values. Isolation Forest separately identifies outliers among seven defined metrics. Both mechanisms can describe the same underlying disturbance and potentially raise more than one alert. Debouncing, incident keys, and resolution grace periods manage the resulting notification volume but do not eliminate all false positives or establish statistical detection accuracy.

The AI Agent uses retrieved metrics and alerts as context for explanations and suggested checks. Its responses must be compared against the underlying queries and current device status. The design does not authorize the language model to assert an unverified root cause or apply corrective configuration commands on behalf of an operator without approval and an appropriate control process.

The assistant does not capture packets and does not replace the IDS. On receiving a question, the Flask service assembles relevant device measurements from Prometheus and recent alerts from PostgreSQL, then sends that context to the Groq API. A response may summarize a trend, propose plausible explanations, or recommend a diagnostic command.

## 2.5 Positioning and comparison with other solutions

### *2.5.1 Reference platforms*

Zabbix is a widely adopted open-source monitoring platform supporting SNMP, native agents, IPMI, and several other collection mechanisms. Its strengths include a mature administration interface, extensive templates, and a large support community. The comparison in this project focuses on the baseline monitoring experience and does not claim that the larger Zabbix ecosystem is fundamentally unable to integrate external security analytics, anomaly detection, or AI-supported investigation.

Grafana combined with Prometheus is a common observability stack, especially in cloud-native environments, and provides powerful time-series visualization. Alerting can be added through Grafana Alerting or related components, but IDS event ingestion and the particular workflow demonstrated by NetMon SOC require additional integration and configuration.

Security Onion is a network-security monitoring distribution that integrates technologies such as Zeek and Suricata with search and analysis tooling. It focuses more heavily on security operations than on conventional network performance metrics. Its deployment and resource requirements can be substantial for a small demonstration environment.

### *2.5.2 Rationale for the NetMon SOC architecture*

***Table 2.4. Comparison of NetMon SOC with reference solutions***

| Feature | NetMon SOC | Zabbix | Grafana + Prometheus | Security Onion |
|---|---|---|---|---|
| SNMP monitoring | Yes | Yes | Yes | Not integrated by default* |
| Node Exporter | Yes | Via agent | Yes | Not integrated by default* |
| IDS integration | Suricata | Not built in* | Not built in* | Zeek + Suricata |
| ML anomaly detection | Isolation Forest | Not built in* | Not built in* | Not built in* |
| AI assistant (LLM) | Groq LLM | Not built in* | Not built in* | Not built in* |
| Multi-channel alerts | Telegram | Email, Slack | AlertManager | Email |
| Alert debounce | Configurable | Yes | Yes | Not built in* |
| Configuration approach | One config.py | Web administration | Additional configuration | Complex setup |
| Infrastructure cost | Low (lab estimate) | Medium (relative) | Low (relative) | High (relative) |
| SMB fit (project view) | Designed for SMB lab | Suitable | Varies by setup | Dedicated security focus |

Note: An asterisk in the comparison table denotes a function considered built-in within the limited scope of this project. It does not indicate a permanent or universal restriction of another product ecosystem.

# CHAPTER 3. IMPLEMENTATION OF NETMON SOC

## 3.1 Preparing the emulation environment

### *3.1.1 Platform, software versions, and device inventory*

The environment runs on a physical host with PNETLab, alongside the NetMon SOC application, Prometheus, and PostgreSQL. The host needs sufficient RAM for the emulated network devices and application services, multiple CPU cores for virtual machines and background workers, and SSD storage to support time-series and relational database activity.

***Table 3.1. Software versions used in the system***

| Component | Version | Role |
|---|---|---|
| PNETLab | Latest (as documented) | Network emulation platform |
| pfSense | 2.7.x (FreeBSD) | Firewall, gateway, and IDS host |
| Cisco IOSv | 15.x | Virtual router and switch images |
| Ubuntu Server | 22.04 LTS | Server01 and Server02 |
| Prometheus | 2.x | Time-series storage and scraping |
| SNMP Exporter | 0.x | SNMP-to-Prometheus exporter |
| Node Exporter | 1.x | Linux host metrics |
| Suricata | 6.x / 7.x | IDS engine on pfSense |
| PostgreSQL | 15.x | Relational database |
| Python | 3.12 | NetMon SOC application runtime |
| Flask | 2.3+ | Web framework |
| scikit-learn | 1.3+ | Isolation Forest implementation |
| ZeroTier | 1.x | Management overlay networking |

***Table 3.2. Monitored devices and their functional roles***

| Device / Type | IP address | Role | Assigned operator |
|---|---|---|---|
| pfSense — Firewall | 172.24.0.115 (ZT) | Firewall, NAT, ZeroTier VPN, Suricata IDS | tuan |

| Device / Type | IP address | Role | Assigned operator |
|---|---|---|---|
| Core-Router — Cisco IOSv router | 10.20.0.1 | OSPF backbone and network-zone interconnection | vuong |
| SwL3A — Cisco IOSv L3 switch | 10.20.0.2 | VLAN10 IT, VLAN20 Sales, DHCP | vuong |
| SwL3B — Cisco IOSv L3 switch | 10.20.0.3 | VLAN30 HR, VLAN40 Engineering, DHCP | tuan |
| Server02 — Ubuntu Linux | 10.20.200.10 | Nginx web service in DMZ1 | vuong |
| Server01 — Ubuntu Linux | 10.20.201.10 | BIND9 DNS and vsftpd FTP in DMZ2 | vuong |

### *3.1.2 Emulated network topology in PNETLab*

The PNETLab topology contains six virtual nodes interconnected by the platform's internal Linux bridges.

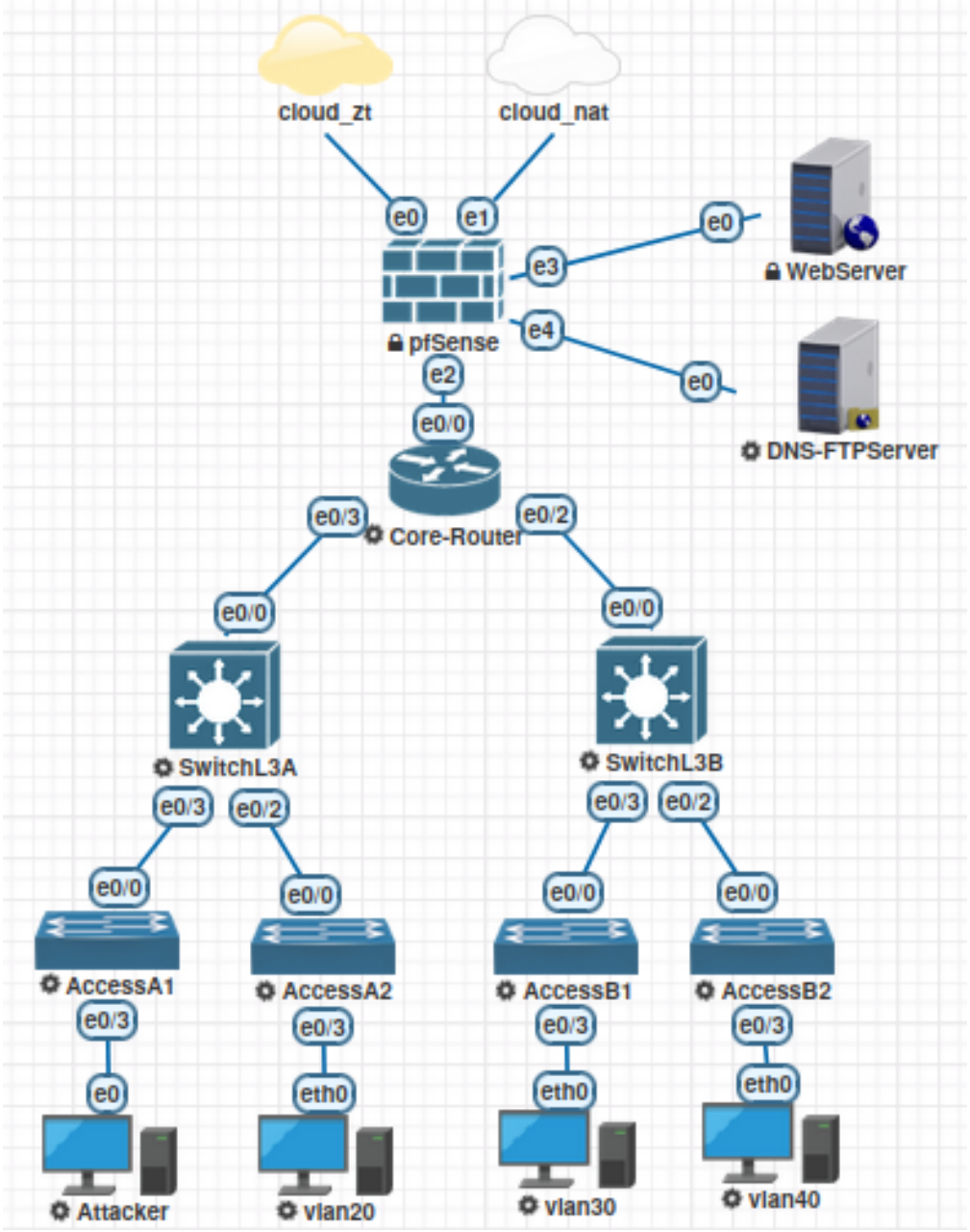



*Figure 3.1. PNETLab network topology used in the experiment*

### *3.1.3 pfSense, security zones, and ZeroTier*

Interface configuration: pfSense has virtual interfaces for WAN, the internal LAN leading to Core-Router, Web-DMZ, DNS-FTP, and the ZeroTier overlay. Interfaces are assigned static addresses according to the network plan.

NAT configuration: Source NAT or masquerading is provided for internal traffic leaving through the WAN interface, allowing lab nodes to reach external package repositories or supporting services under the firewall's routing policy.

Firewall rules: The planned access model follows the principle of least privilege. User VLANs can reach the intended web DMZ ports (80 and 443) and DNS service (port 53) under explicitly configured policies. Direct communication between unrelated user VLANs is restricted, and inbound traffic from WAN to DMZ services is tightly controlled.

SNMP configuration: The firewall exposes SNMPv2c monitoring data using a community value maintained in the deployment configuration. Prometheus SNMP Exporter accesses pfSense through the ZeroTier address 172.24.0.115 and uses the if_mib module to gather interface measurements.

Suricata configuration: The IDS package is installed through pfSense Package Manager and enabled in detection-only mode on the five monitored interfaces. Emerging Threats Open is used as the principal rule source. EVE JSON output is configured to include the relevant alert, DNS, HTTP, and TLS records.

ZeroTier configuration: The ZeroTier client on pfSense joins the lab's overlay network, and 172.24.0.115 is assigned to the firewall through ZeroTier management. A firewall rule permits the authorized overlay management peer to establish SSH sessions for Suricata log collection.

## 3.2 Establishing the routed core and test services

Infrastructure nodes have distinct purposes: the core router connects network segments, Layer 3 switches serve user VLANs, and web, DNS, and FTP services provide endpoints for availability and security testing.

### *3.2.1 Core router and OSPF routing*

Core-Router uses Cisco IOSv and the Open Shortest Path First (OSPF) routing protocol to distribute routes between the internal segments. Its relevant interfaces connect to pfSense, SwL3A, SwL3B, and routed paths serving the DMZs.

The OSPF configuration places the internal routed links in area 0 to keep the small laboratory backbone straightforward. An SNMP community is also configured on the router so that Prometheus SNMP Exporter can obtain interface and status counters.

```
! Cấu hình SNMP cơ bản trên Cisco IOSv
snmp-server community <community-string> RO
snmp-server ifindex persist


! Cấu hình OSPF
router ospf 1
 network 10.20.0.0 0.0.255.255 area 0
 passive-interface default
 no passive-interface GigabitEthernet0/0  ! Interface kết nối pfSense
```

### *3.2.2 VLANs and DHCP on the Layer 3 switches*

SwL3A and SwL3B are Cisco IOSv Layer 3 switching nodes responsible for access to the user networks and the corresponding DHCP services.

SwL3A serves VLAN10 (10.20.10.0/24, IT) and VLAN20 (10.20.20.0/24, Sales), with an SVI and a DHCP pool for each subnet. SwL3B provides the same functions for VLAN30 (10.20.30.0/24, HR) and VLAN40 (10.20.40.0/24, Engineering).

SNMPv2c monitoring is enabled on both Layer 3 switches, using the same collection model as Core-Router. Prometheus then observes port operational state and traffic counters through SNMP Exporter.

### *3.2.3 Linux servers providing web, DNS, and FTP*

Server02 (10.20.200.10 — Web Server): This Ubuntu Server 22.04 LTS node runs Nginx and serves a simple HTTP application from DMZ1. It provides an endpoint for connectivity checks and controlled web-security tests.

Server01 (10.20.201.10 — DNS and FTP): This Ubuntu Server 22.04 LTS node runs BIND9 for the internal lab.local namespace, including an example mapping of web.lab.local to 10.20.200.10, and vsftpd for an FTP service in DMZ2. Node Exporter also listens on port 9100.

```
# Tải Node Exporter

wget
https://github.com/prometheus/node_exporter/releases/download/v1.7.0/node_e
xporter-1.7.0.linux-amd64.tar.gz

tar xvf node_exporter-*.tar.gz



# Cài đặt service systemd

sudo cp node_exporter /usr/local/bin/

sudo useradd -rs /bin/false node_exporter



# /etc/systemd/system/node_exporter.service

sudo systemctl enable --now node_exporter
```

## 3.3 Implementing the metrics and IDS pipelines

### *3.3.1 Prometheus and SNMP Exporter*

Prometheus and SNMP Exporter are installed on the host that also runs PNETLab. The prometheus.yml configuration defines the six scrape jobs described in the design and sets scrape_interval: 15s together with evaluation_interval: 15s.

```
global:
  scrape_interval:     15s
  evaluation_interval: 15s
scrape_configs:
  - job_name: "snmp-pfsense"
    metrics_path: /snmp
    params:
      module: [if_mib]
      target: ["172.24.0.115"]
    static_configs:
      - targets: ["localhost:9116"]
        labels: {device: "pfSense", role: "firewall"}
  - job_name: "node-server02"
    static_configs:
```

SNMP Exporter listens on port 9116 and acts as a bridge between SNMP data from the network appliances and the Prometheus metrics format. The if_mib module is used to expose IF-MIB interface counters and operational states.

### *3.3.2 Node Exporter and endpoint verification*

Node Exporter is installed on Server01 and Server02 and exposes a /metrics endpoint on port 9100. The Prometheus target page at http://localhost:9090/targets is used to verify that the jobs node-server01 and node-server02 report UP.

### *3.3.3 Suricata IDS and EVE JSON ingestion*

Suricata is installed through pfSense WebGUI under System → Package Manager → Available Packages. The sensor instances and their logging options are then managed from Services → Suricata.

Interface setup: WAN, Web-DMZ, DNS-FTP, LAN, and ZeroTier are each added to the monitored-interface list. Each interface has its own configuration and logging path.

EVE JSON logging: The logging settings for each interface enable structured output for alert, dns, http, tls, and flow events. The resulting files follow a per-interface path convention such as /var/log/suricata/suricata_<interface_id>/eve.json.

Rule set: Emerging Threats Open rules are updated from the configured repository. Enabled categories in the lab include emerging-attack_response, emerging-exploit, emerging-malware, emerging-scan, emerging-web_client, and emerging-web_server.

Operational verification: After the Suricata service starts, its status is checked on the pfSense console together with the expected log files.

```
ps aux | grep suricata

tail -f /var/log/suricata/suricata_vtnet125952/eve.json
```

SSH authentication: A key pair is created on the NetMon host, and the public key is registered for the permitted pfSense management account through System → User Manager → Users → Authorized SSH Keys. The connection is verified without interactive password entry.

After each Suricata interface is enabled, validation checks more than the service's running state. The relevant EVE JSON file should contain new events with valid event_type and timestamp fields; the worker should resume from the correct read position; and the database should not duplicate a record after worker restart. If the management connection is interrupted, event occurrence and ingestion times may differ, so the original timestamp should be preserved for accurate SOC timelines.

## 3.4 Backend, anomaly models, and alert processing

NetMon SOC source code is organized by functional module. After dependencies and environment variables are configured, the application initializes the database, registers background collection tasks, and serves the web interface. API keys, notification tokens, and connection passwords remain deployment-specific values and are not reproduced in the report.

### *3.4.1 Initializing Flask and the database*

```
cd /opt/netmon

python3 -m venv venv

source venv/bin/activate

pip install -r requirements.txt
```

Environment configuration: A .env file is created from the .env.example template, and the deployment-specific connection settings, tokens, and application values are supplied.

```
DATABASE_URL=postgresql://netmon:netmon_pass@localhost:5432/netmon

PROMETHEUS_URL=http://localhost:9090

TELEGRAM_BOT_TOKEN=<bot_token>

TELEGRAM_VUONG_ID=<chat_id_vuong>

TELEGRAM_TUAN_ID=<chat_id_tuan>

GROQ_API_KEY=<groq_api_key>

SURICATA_SSH_HOST=172.24.0.115

SURICATA_SSH_USER=admin

SURICATA_SSH_KEY=/home/minhvuong/.ssh/id_rsa
```

```
# Tạo PostgreSQL database và user

sudo -u postgres psql -c "CREATE USER netmon WITH PASSWORD 'netmon_pass';"

sudo -u postgres psql -c "CREATE DATABASE netmon OWNER netmon;"


# Chạy script setup tự động

bash scripts/setup_db.sh
```

```
python app.py

# Output:

# [env] Đã load 12 biến từ /opt/netmon/.env

# [netmon-soc] http://0.0.0.0:5000

# [worker:metrics] started

# [worker:suricata] started

# [worker:checker] started
```

Once the application is started, the web interface is accessible at http://<host-ip>:5000 in the laboratory. The landing page presents monitored devices and team information.

### *3.4.2 Training and loading Isolation Forest models*

Training begins after the monitoring pipeline has accumulated a stable period of observations: at least 24 hours for basic verification and ideally the configured seven-day history. The model-training process retrieves samples from Prometheus, applies the selected feature transformation, and serializes the resulting models.

```
cd /opt/netmon

python scripts/train_models.py
```

The training script selects features according to device class. Traffic rates and interface errors are used for network devices; CPU, RAM, and disk utilization are used for Linux hosts. The documented query window is seven days (168 hours) at five-minute spacing, giving approximately 2,016 observations per metric where collection coverage is complete.

After successful training, seven model artifacts are stored in the models/ directory, corresponding to the documented metrics.

```
models/
├── isolation_forest_cpu_percent.pkl          (628 KB)
├── isolation_forest_mem_used_pct.pkl        (1.28 MB)
├── isolation_forest_disk_used_pct.pkl       (1.40 MB)
├── isolation_forest_traffic_in_bps.pkl      (1.32 MB)
├── isolation_forest_traffic_out_bps.pkl     (1.25 MB)
├── isolation_forest_in_errors.pkl           (216 KB)
└── isolation_forest_out_errors.pkl          (119 KB)
```

Each serialized artifact reflects the fitted estimator and associated StandardScaler. The documented forest contains 200 trees, and joblib.dump is used for persistence.

The application is restarted after model creation so that the AI service can load the current artifacts and activate the anomaly-evaluation path.

```
python app.py

# [ai] Loaded model: cpu_percent

# [ai] Loaded model: mem_used_pct

# ...
```

The historical Prometheus window serves as the training baseline. Before fitting, the operator should check for missing samples, excessively short histories, and traffic or resource changes caused by earlier lab tests. Otherwise, the detector may learn the test

conditions as normal behavior. Large artificial spikes help confirm that the ingestion-to-model-to-alert path is connected, but they do not establish sensitivity to small, gradual, or unfamiliar anomalies.

#### *3.4.3 Notifications and alert-state management*

The application evaluates each incoming measurement against its configured threshold and the preceding state of the associated condition. Once enough consecutive failures are observed, an alert is created or updated and a notification is routed to the responsible recipient. When the triggering condition recovers, a stability interval is observed before automatic resolution. The laboratory settings include three-sample debouncing, a 300-second resolution grace period, and reminders after 600 seconds.

A useful notification includes at least the event type, device or endpoint, triggering metric or signature, time, and current processing state. An incident key helps recipients distinguish a new issue from a reminder or a recovery message. Leader/member routing limits who receives particular notifications; it is not equivalent to web-application access control, so the two security concerns must be treated independently.

### 3.5 Operator interface and workflow

#### *3.5.1 Device dashboard and alert-management view*

The main dashboard provides a near-real-time overview of network infrastructure. Its data are refreshed through AJAX requests to /api/devices at REFRESH_INTERVAL=15 seconds.

The summary area displays four aggregate measures: total monitored devices, online devices, down interfaces, and services reporting problems. Status colors distinguish normal conditions, warnings, and critical conditions, using green, amber, and red.

Each device appears as a card showing name, type, IP address, online/offline status, uptime, interface state, and traffic. A detailed view expands the recent traffic history to a 60-minute chart.

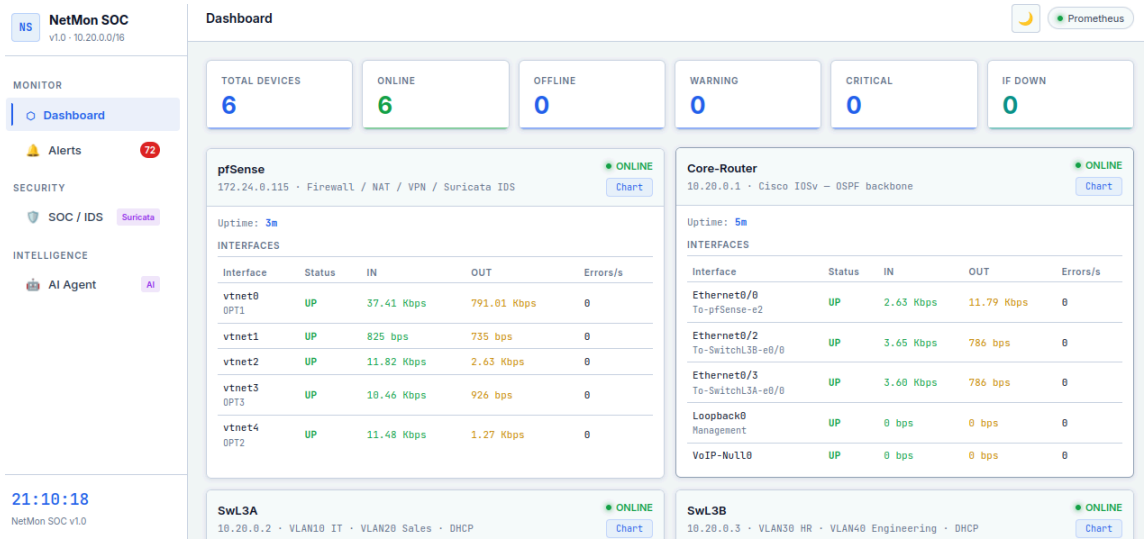



*Figure 3.2. Real-time device-monitoring dashboard*

The Alerts page provides a consolidated view of application-generated alerts with filters by severity (critical, warning, information), processing state (open or resolved), device, and time range.

Summary widgets display total alerts, critical and warning counts, AI anomaly alerts, and unresolved items. The detailed table includes timestamp, device, severity, category, message, and state. A Resolve action lets an operator mark an incident as handled, optionally recording a note. A timeline chart shows how alert volume changes across the selected period, supporting follow-up review and prioritization.

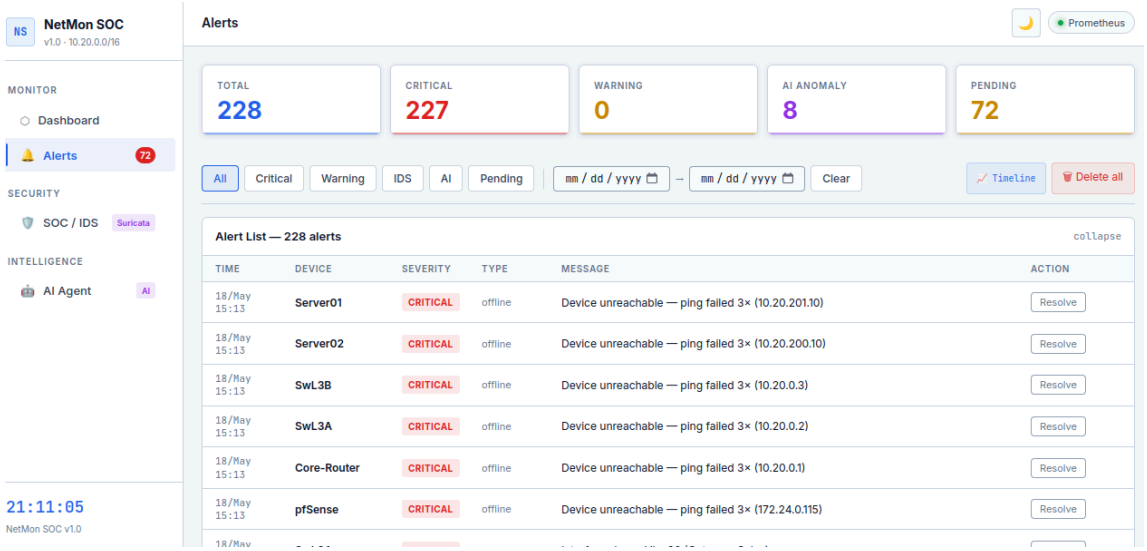



*Figure 3.3. Alert management interface*

### *3.5.2 SOC dashboard and AI Agent*

The SOC page is the dedicated security-analysis interface. It aggregates IDS events retrieved from Suricata on the five monitored pfSense interfaces and provides a centralized view for examining source addresses, target services, signatures, and severity.

The overview summarizes activity during the preceding 24 hours, including total events, critical and warning counts, frequently observed source addresses, and distribution by network zone. An hourly timeline chart helps identify periods with concentrated detections.

The detailed event table supports filters by severity, detection category, source IP, interface, and network zone. Each record includes timestamp, source IP and port, destination IP and port, protocol, triggering signature, and classification. Selecting a source address opens an analysis panel with its event count, category distribution, targeted destination addresses, and activity history, facilitating a focused review of repeated observations.

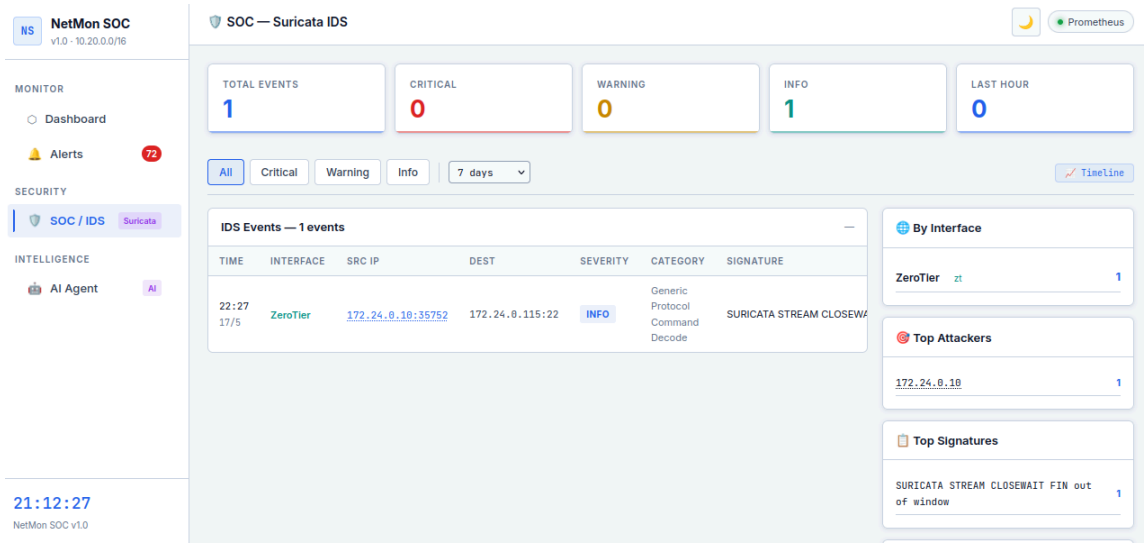



***Figure 3.4. SOC dashboard for Suricata IDS events***

The AI Agent page provides a messenger-style conversational interface powered by a Groq-hosted language model. Operators can enter natural-language questions, read responses in a scrolling conversation, and request follow-up explanations.

For each question, the application assembles context from current Prometheus device status, near-real-time measurements, the ten most recent database alerts, and an

infrastructure overview. This context is placed in the request prompt before the Groq API call.

Typical queries include "What is Server02's current CPU utilization?", "Which devices have active alerts?", "Explain the interface-down alert on SwL3A," or "Which command checks an OSPF neighbor on Core-Router?" Anomaly events can also provide context for suggested causes or next checks.

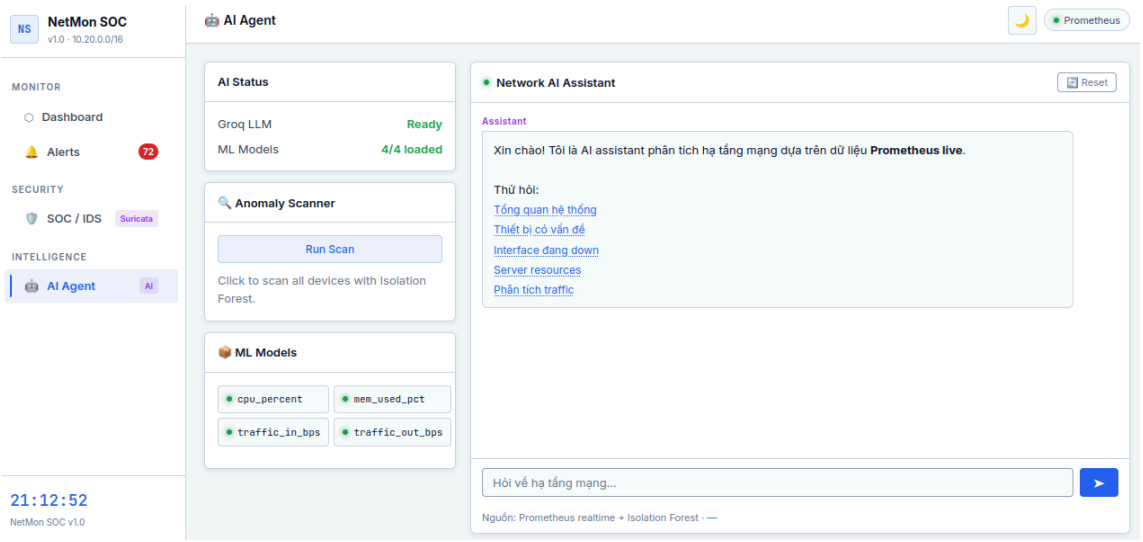



***Figure 3.5. AI Agent interface with live network context***

Notification validation compares the persisted alert record with the Telegram message actually received, then introduces a recovery condition to confirm the resolved transition. For the AI Agent, a response is meaningful only if its cited metric values can be checked against Prometheus for the corresponding moment. Both checks illustrate the broader principle that operator-facing information must remain consistent with the underlying evidence pipeline.

# CHAPTER 4. TESTING, RESULTS, AND ANALYSIS

## 4.1 Test methodology and scope

### *4.1.1 Environment, tools, and evaluation rules*

Tests were performed against the deployed PNETLab environment. A Kali Linux machine connected to the laboratory through ZeroTier served as a controlled traffic-generation client.

stress-ng: generate controlled CPU and memory load on a test server.

iperf3: generate and measure traffic between permitted lab endpoints.

### *4.1.2 Test scenario inventory*

***Table 4.1. Summary of the primary validation scenarios***

| Code | Scenario / Tool | Target | Expected outcome |
|---|---|---|---|
| TC01 | Device offline / Power off VM | pfSense, router, switch | Device-offline alert after three failed ping checks |
| TC02 | Interface down / Disable interface | SwL3A GE0/1 | Interface-down alert |
| TC03 | High CPU / stress-ng | Server02 | CPU >85% alert and AI anomaly |
| TC04 | High RAM / stress-ng | Server01 | RAM >90% threshold and AI anomaly (test objective) |
| TC05 | Disk full / Create large file | Server02 | Disk >85% alert |
| TC06 | Port scan / Nmap | Server01, Server02 | Suricata Port Scan event |
| TC07 | SQL injection / SQLMap | Server02 (Nginx) | Suricata Web Attack event |
| TC08 | Directory traversal / curl | Server02 | Suricata Web Attack event |
| TC09 | SSH brute force / Hydra | Server01 | Suricata Brute Force event |
| TC10 | Low-intensity DoS / hping3 | pfSense WAN | Suricata DoS event |

| Code | Scenario / Tool | Target | Expected outcome |
|---|---|---|---|
| TC11 | Traffic anomaly / iperf3 flood | Router/switch | AI anomaly alert |
| TC12 | Telegram message / TC01 | Entire system | Telegram notification delivered |
| TC13 | Auto-resolution / Recover TC01 | pfSense | Alert automatically resolved after five minutes |

Thirteen principal validation scenarios cover the main NetMon SOC workflows: infrastructure status, resource measurements, IDS observation, machine-learning outliers, and alert-life-cycle transitions. The WAN, Web, FTP, and DNS traffic-generator menus provide additional variants; they are not counted as additional scenarios within the primary set of thirteen. Each test is reviewed through three linked artifacts: the generated stimulus, the evidence acquired by the collection pipeline, and the resulting display or notification.

## 4.2 Infrastructure-health monitoring tests

Two types of condition were tested separately: loss of device or interface connectivity, and elevated utilization while a device remained operational. Detection latency is interpreted as the time from stimulus through the next polling or log-reading opportunity, evaluation of alert conditions, and notification.

### *4.2.1 Offline devices and disabled interfaces*

Procedure: The pfSense virtual machine was powered off within PNETLab. The team observed the monitoring system across three ping-check cycles, covering approximately 45 seconds, and checked the device state, alert record, and outbound notifications.

Observed result: After three consecutive failed checks, an alert stating that pfSense did not respond to ICMP ping was recorded with severity="critical". Telegram notifications reached both the leader, Vuong, and the assigned member, Tuan. In the dashboard, the pfSense card changed to the offline state and used the red critical-state indicator.

After pfSense was restarted, the subsequent check recognized that the device had returned online, approximately one 15-second cycle later. A 300-second recovery grace

period then began. Once the healthy condition remained stable for five minutes, the alert was automatically resolved and a recovery notification was sent through Telegram.

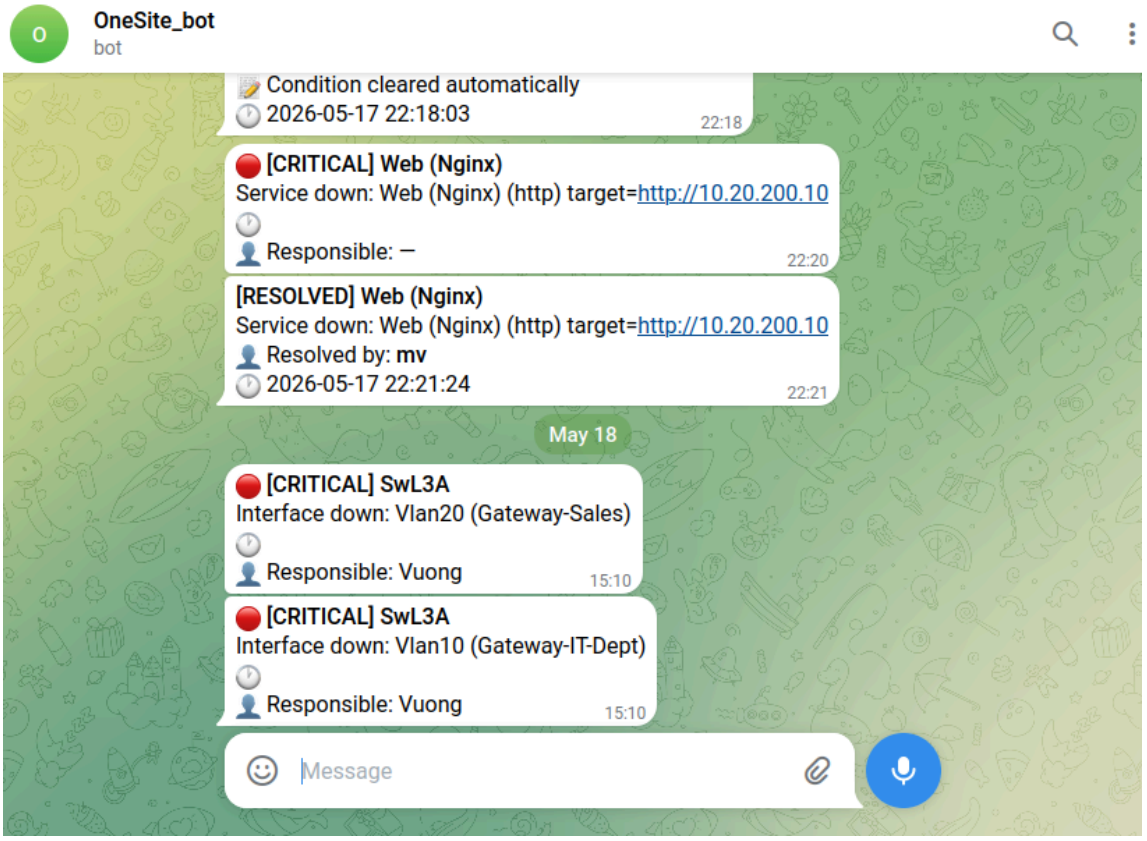



*Figure 4.1. Telegram alerts for pfSense outage and automatic recovery*

Procedure: On SwL3A, GigabitEthernet0/1, labeled VLAN10-Uplink, was administratively shut down.

```
SwL3A(config)# interface GigabitEthernet0/1

SwL3A(config-if)# shutdown
```

Observed result: Within roughly 30 seconds, or two subsequent scrape cycles, SNMP Exporter collected ifOperStatus=2 (down) for that interface. The logic in alerting.py recognized the interface condition and generated a warning alert describing GE0/1 (VLAN10-Uplink) on SwL3A as DOWN. The corresponding Telegram message was routed to Vuong as the responsible member. The finding is specific to the status and polling behavior observed in the lab.

These tests use different detection mechanisms. Offline status is established after a sequence of failed ping checks, whereas a down interface is identified through the

SNMP operational and administrative status indicators. Their different response times follow from their trigger conditions, polling schedules, and evaluation logic.

### *4.2.2 CPU, memory, and disk-utilization tests*

CPU test: The team used stress-ng on Server02 to increase CPU utilization beyond the configured 85-percent threshold.

stress-ng --cpu 4 --timeout 120s

Observed CPU result: About 30–45 seconds after utilization passed the configured threshold, the system created an alert with the recorded message "CPU Server02 = 92.3% (threshold 85%)." The Isolation Forest module also marked the same period as anomalous, with score = −0.68, below the laboratory critical threshold of −0.55, generating a separate critical AI anomaly alert for cpu_percent.

In the memory test, stress-ng was used with --vm 1 --vm-bytes 80% to raise utilization toward approximately 80 percent. This remained below the configured RAM threshold of greater than 90 percent, so a threshold alert was not justified by that observation alone. An Isolation Forest model may still flag a substantial deviation from its learned baseline, but any such result must be reported separately from the threshold outcome in the associated evidence table.

***Table 4.2. Results of server-resource threshold testing***

| Scenario | Measured value / Threshold | Detection time | Alert type |
|---|---|---|---|
| CPU stress on Server02 | Observed: 92.3% / Limit: 85% | ~30 seconds | Threshold + AI anomaly |
| RAM stress on Server01 | Observed: 80.2% / Limit: 90% | N/A | AI anomaly only |
| Disk capacity on Server02 | Observed: 87.5% / Limit: 85% | ~15 seconds | Threshold |
| Normal CPU operation | Observed: 12.4% / Limit: — | — | No alert |

## 4.3 Intrusion-detection tests by network zone

IDS testing is grouped by network zone to make clear where traffic originated, which pfSense interface observed it, and how the resulting event appeared in the application. Nmap, SQLMap, Hydra, hping3, and the project's controlled request generators were used only against designated laboratory systems.

### *4.3.1 WAN edge, port scanning, and low-rate denial-of-service patterns*

This group generated port-discovery patterns and abnormal traffic at the WAN boundary to exercise the Suricata sensor on pfSense. Evaluation focused on the triggered signature, source and destination information, event timestamp, and matching SOC Dashboard record.

The NetMon Attack Console organizes controlled traffic generation by exposed service surface. The WAN group covers scan-like patterns and elevated traffic; the Web group sends test requests containing unusual payload structures; the FTP group examines authentication and service commands; and the DNS group exercises query frequency, label length, and record types.

Procedure: From the Kali Linux host connected to the lab VLAN10, a network scan was directed at Server01 to examine whether Suricata recognized reconnaissance behavior.

```
nmap -sV -sC -p- --min-rate=1000 10.20.201.10
```

Observed result: The Suricata instance monitoring the LAN interface recorded several events in the Port Scan or Detection of a Network Scan categories. Approximately 10–20 seconds later, the SOC worker retrieved relevant records and stored them in soc_events. The dashboard grouped events associated with the Kali source address and displayed their attack category and configured severity, confirming end-to-end ingestion and presentation for this test.

Procedure: A low-intensity SYN-flood-style traffic pattern was sent toward the pfSense WAN interface to test whether the corresponding rule set generated an observable event.

```
hping3 -S --flood -V -p 443 172.24.0.115
```

Observed result: Suricata on the WAN interface, identified as zone wan, recorded an "Attempted Denial of Service" classification associated with a SYN-flood signature. Because the load was kept low and the environment was isolated, the experiment did not demonstrate significant impact on pfSense availability.

The NetMon test console provides several categories of controlled exercises: WAN and pfSense boundary traffic, web-server requests, FTP service activity, and DNS query patterns. Each category includes multiple representative variants. These menus expand the types of evidence that can be compared with Suricata logs while keeping the primary evaluation organized by protocol and observation zone instead of listing each payload as a separate project capability.

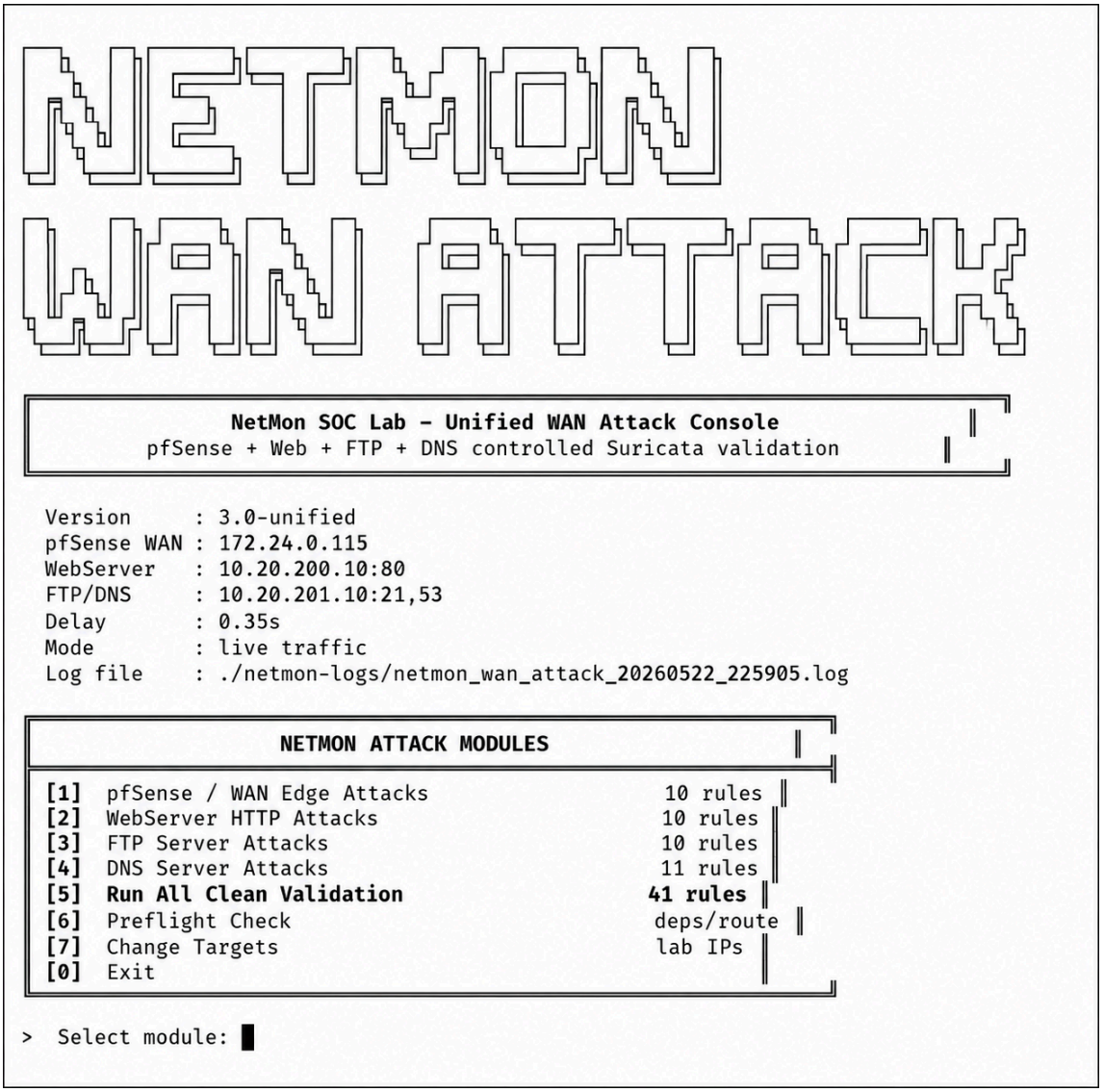



*Figure 4.2. Main NetMon WAN Attack Console interface*

The console's main screen shows practical context such as the application version, pfSense WAN address, test web-server address, FTP/DNS service address, active traffic mode, and output log path. Available modules include WAN Edge Attacks, WebServer HTTP Attacks, FTP Server Attacks, DNS Server Attacks, and Run All Clean Validation. The menu structure makes it easier to repeat one selected group and compare the corresponding event sequence against Suricata records.

WAN-edge exercises focus on activity that encounters pfSense before reaching an internal or DMZ segment. The firewall is a particularly important observation point because network access policies and IDS visibility overlap there.

Representative WAN scenarios include service discovery, responses to port probes, unusual traffic generation, SYN bursts, and low-intensity denial-of-service-like patterns.

When the enabled signatures match the observed traffic, Suricata logs the events and the SOC worker imports and normalizes them for display.

```
hping3 -S -p 443 --flood 172.24.0.115
nmap -sS -Pn -p 1-1000 172.24.0.115
```

### *4.3.2 Web-application attack indicators*

The web-service exercises included SQL injection-like requests generated with SQLMap and controlled directory-traversal patterns.

```
sqlmap -u "http://10.20.200.10/?id=1" --dbs --batch
curl "http://10.20.200.10/../../../../etc/passwd"
curl "http://10.20.200.10/admin/../../../etc/shadow"
```

Observed result: The Web-DMZ Suricata sensor, zone dmz-web, logged events in the "Web Application Attack" category, including signatures resembling "ET WEB_SERVER SQL Injection Attempt" and "ET WEB_SERVER Possible Directory Traversal." Their application severities ranged from warning to critical according to rule priority. The events retained HTTP URL and method information, helping the SOC interface describe the exact request patterns that were observed.

The Web Attack Generator also groups several controlled web-application patterns, including repeated login attempts, SQL injection, cross-site scripting, command-injection indicators, path traversal or local-file inclusion patterns, webshell-access probes, file-upload scenarios, directory enumeration, and light DoS/SYN burst tests.




*Figure 4.3. Web Attack Generator interface*

```
curl "http://10.20.200.10/?id=1' OR '1'='1"
curl "http://10.20.200.10/search?q=<script>alert(1)</script>"
curl "http://10.20.200.10/../../../../etc/passwd"
curl "http://10.20.200.10/admin/"
```

The tests showed that Suricata produced multiple web-application event categories within the enabled rule set. Records included URLs, HTTP methods, source and destination addresses, and signatures. These fields support the SOC timeline, source-IP summaries, and follow-up forensic review of what was sent to the web server.

### *4.3.3 FTP and DNS services and combined IDS results*

```
hydra -l root -P /usr/share/wordlists/rockyou.txt -t 4 ssh://10.20.201.10
```

Observed result for the authentication-related exercise: Suricata logged signatures such as "ET SCAN Potential SSH Scan" and "SURICATA STREAM 3way handshake anomaly." Many attempts within a short period formed a recognizable cluster on the SOC timeline. The Telegram notification was categorized as critical in the reported scenario because the relevant Suricata event had priority 1.

The FTP testing menu includes controlled scenarios for repeated authentication attempts, anonymous-login checks, FTP bounce behavior, suspicious uploads, path traversal, bulk file operations, SITE commands, hidden-file uploads, and directory enumeration.

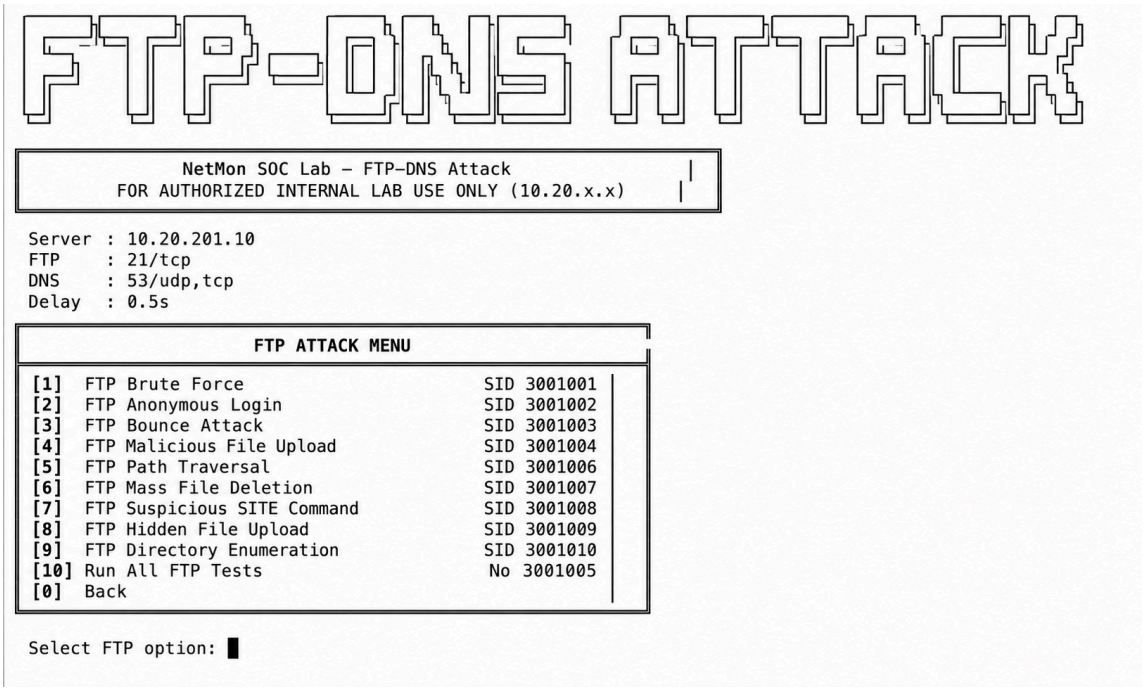



*Figure 4.4. FTP Attack Menu interface*

The FTP group broadens the monitored application surface beyond web and SSH traffic. File-transfer services can expose authentication and configuration risks, particularly where anonymous access or poorly protected credentials are permitted.

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt ftp://10.20.201.10
ftp 10.20.201.10
curl ftp://anonymous:anonymous@10.20.201.10/
curl -T shell.php ftp://user:password@10.20.201.10/uploads/
```

Expected behavior is for matching Suricata rules to record events related to repeated logins, suspicious transfers, or enumeration attempts. Such records are imported into the SOC and classified as FTP-related activity or Brute Force according to the signature that fired.

The DNS exercise menu covers representative query patterns: high-frequency UDP requests, AXFR attempts, unusually long query labels, ANY and TXT queries, subdomain enumeration, NULL queries, CHAOS version.bind, PTR reverse lookups, and encoded-domain patterns.

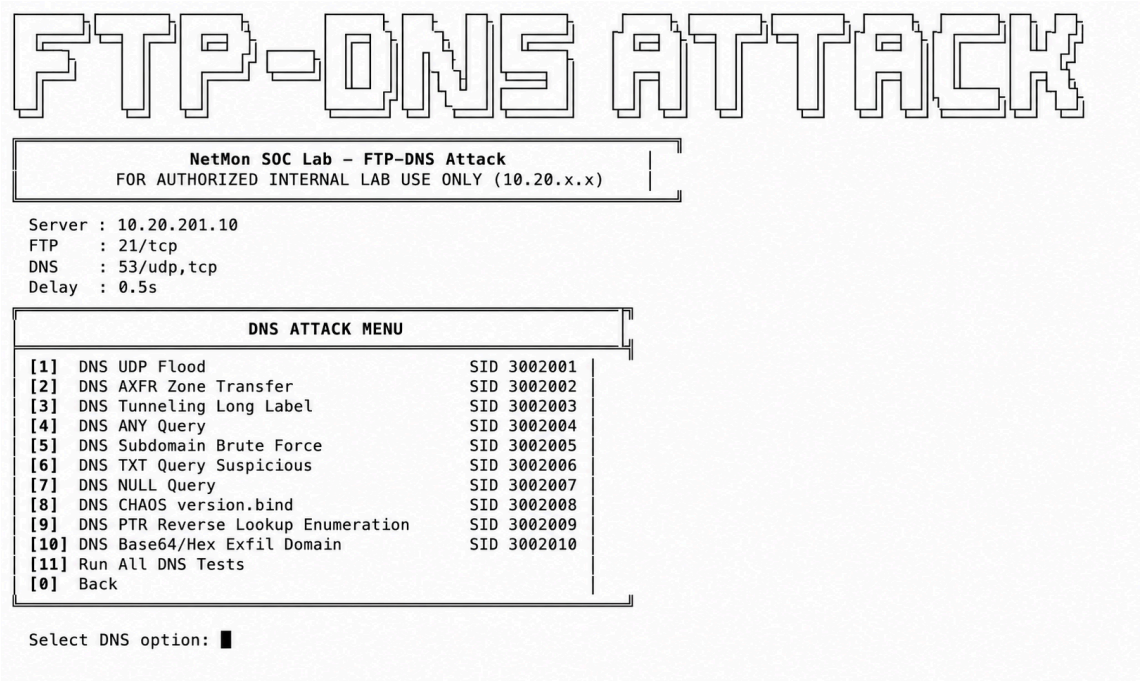



***Figure 4.5. DNS Attack Menu interface***

DNS exercises extend the analysis from web and file-transfer traffic to core application infrastructure. Requests containing long labels, unusual record types, or repeated query structures are generated to test whether the sensor and ingestion pipeline record their characteristics.

```
dig @10.20.201.10 axfr lab.local
dig @10.20.201.10 ANY lab.local
dig @10.20.201.10 TXT suspicious.lab.local
dig @10.20.201.10 $(echo "secretdata" | base64).lab.local
```

For the DNS scenarios exercised, the dashboard recorded events corresponding to the generated query patterns. Reliable classification still requires inspecting the triggering signature, legitimate service context, and sufficient traffic history.

*Table 4.3. Suricata IDS validation results after additional scenarios*

| No. | Scenario / Tool | Zone and simulated behavior | Recorded outcome |
|---|---|---|---|
| 1 | Port scan / Nmap | LAN / Port discovery and service enumeration | Suricata Port Scan / Network Scan events |
| 2 | SQL injection / SQLMap, curl, NetMon Web Attack | Web-DMZ / Abnormal SQL-like request content | Web Application Attack events recorded |
| 3 | Directory traversal / LFI / curl, NetMon Web Attack | Web-DMZ / Attempts to access sensitive paths | Path Traversal / LFI events recorded |
| 4 | XSS simulation / NetMon Web Attack | Web-DMZ / Script-like payload in a request | Web Attack event recorded |
| 5 | Command injection / NetMon Web Attack | Web-DMZ / Command-like request payload | Web Application Attack event recorded |
| 6 | Webshell / File upload / NetMon Web Attack | Web-DMZ / Suspicious upload or file-access request | Abnormal HTTP event recorded |
| 7 | SSH brute force / Hydra | LAN / Server zone / Repeated password attempts | Cluster of scan / brute-force events |
| 8 | FTP brute force / Hydra, NetMon FTP Attack | FTP-DNS zone / Repeated FTP login attempts | FTP brute-force events recorded |

| No. | Scenario / Tool | Zone and simulated behavior | Recorded outcome |
|---|---|---|---|
| 9 | FTP anonymous login / ftp, curl, NetMon FTP Attack | FTP-DNS zone / Anonymous-login attempt | Suspicious FTP access recorded |
| 10 | FTP upload / Directory enumeration / curl, ftp, NetMon FTP Attack | FTP-DNS zone / Uploads and directory listing | FTP Attack / suspicious activity recorded |
| 11 | DNS AXFR transfer / dig, NetMon DNS Attack | FTP-DNS zone / Zone-transfer request | Suspicious DNS / attack-pattern event |
| 12 | DNS tunneling / Long label / dig, NetMon DNS Attack | FTP-DNS zone / Unusual domain-label patterns | DNS tunneling-like pattern recorded |
| 13 | DNS ANY / TXT / NULL / dig, NetMon DNS Attack | FTP-DNS zone / Unusual record-type requests | Suspicious DNS query event |
| 14 | DNS exfiltration simulation / dig, NetMon DNS Attack | FTP-DNS zone / Encoded-domain test patterns | Abnormal query-domain pattern recorded |
| 15 | Low-rate DoS / SYN flood / hping3, NetMon WAN/Web Attack | WAN / Web-DMZ / SYN bursts and low-rate DoS | Attempted Denial of Service event |

A failed FTP login is not, on its own, sufficient to characterize brute force. Likewise, DNS TXT requests or long labels can occur during legitimate operation. IDS records should therefore be treated as indicators that prompt further checking against packet captures, application logs, and host activity.

### 4.4 Machine-learning and alert-life-cycle tests

The machine-learning experiments distinguish deterministic threshold alerts from statistical outliers identified using historical metrics. CPU, memory, and traffic workloads were modified in a controlled manner, after which the model score, alert record, and dashboard state were inspected. An anomaly score shows departure from the learned metric baseline; it does not assign a verified security cause or establish that a particular process is malicious.

### *4.4.1 Network-traffic anomalies*

Procedure: iperf3 was used to produce an unusually large traffic flow between two authorized laboratory servers.

```
# Trên Server01 (iperf server)

iperf3 -s

# Từ máy Kali (iperf client)

iperf3 -c 10.20.201.10 -b 500M -t 60
```

Observed result: The generated traffic reached approximately 500 Mbps, far above the typical baseline of less than 10 Mbps for the monitored lab router. After traffic_in_bps was collected and evaluated, Isolation Forest returned score = −0.71, below the configured critical threshold of −0.55. The application created a critical alert identifying an anomaly in traffic_in_bps and recording the score.

### *4.4.2 CPU/RAM changes and layered alerting*

```
# Tải CPU đột biến trên Server02

stress-ng --cpu 4 --timeout 180s &

# Đồng thời tải RAM

stress-ng --vm 2 --vm-bytes 75% --timeout 180s &
```

During a combined-load test on Server02, CPU utilization rose from approximately 8% to 96.4%, and memory use increased from roughly 35% to 87.2%. The model reported outlier scores of −0.69 for CPU and −0.42 for RAM; the application's configured severity thresholds classified the resulting signals differently. This was a deliberately generated workload, not evidence of malware.

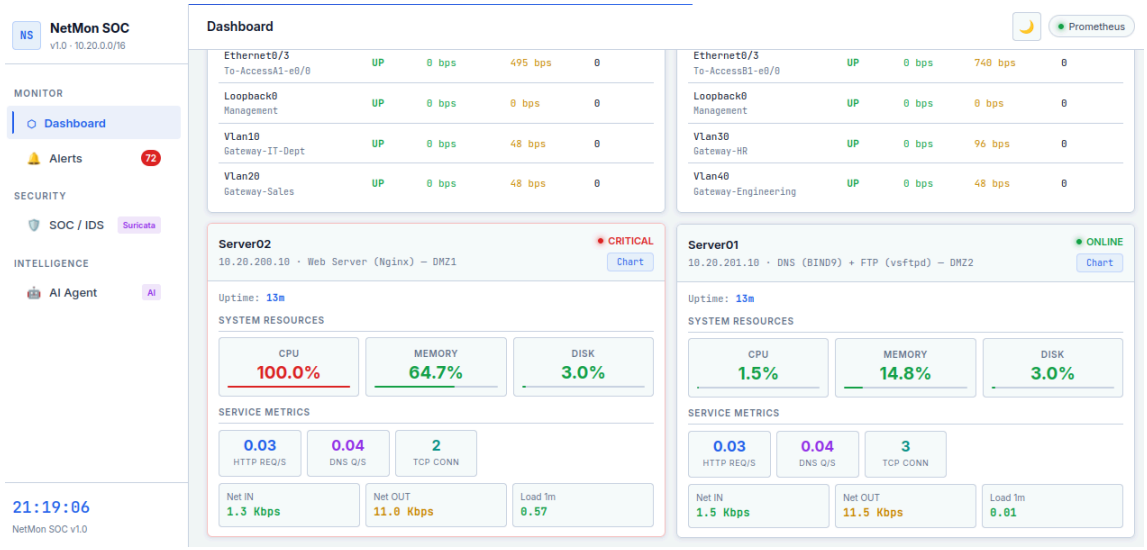



***Figure 4.6. AI anomaly alerts for Server02 CPU and memory utilization***

### *4.4.3 Telegram, debouncing, reminders, and auto-resolution*

Scenario: A high-CPU condition was triggered on Server02 while Telegram messages were monitored on two recipient devices: Vuong, configured as leader, and Tuan, configured as member.

Observed result: Vuong received a detailed notification specifying the alert category, affected device, observed value, occurrence time, and a suggested investigation direction. Tuan received a shorter critical-category notification although Server02 was not his assigned device. The transmitted message used valid HTML formatting with special characters escaped.

Reminder test: After an alert remained active without intervention, the application sent a Telegram reminder after 600 seconds with a message indicating that the incident was still unresolved. Reminders were scheduled every ten minutes until the alert was closed.

Debounce test: CPU load was raised to about 90 percent for only 20 seconds, shorter than the three 15-second cycles required for confirmation. No alert was created because fail_count never reached three.

Auto-resolution test: A sustained high-CPU condition first created an alert after the required failing cycles. Once CPU load returned to normal, fail_count was reset and grace_deadline was set 300 seconds ahead. After five stable minutes, the alert changed

to resolved with resolve_mode="auto" and resolve_by="system," and a Telegram recovery message reached the leader.

*Table 4.4. Results of alert life-cycle validation*

| Scenario | Expected behavior | Observed outcome | Result |
|---|---|---|---|
| 20-second CPU spike | No alert because of debounce | No alert created | Pass |
| Sustained CPU load >45 s | Alert after three failed checks | Alert created after ~45 s | Pass |
| Reminder after ten minutes | Telegram reminder sent | Reminder message received | Pass |
| CPU utilization recovers | Auto-resolution after five minutes | Resolved after ~5 min | Pass |
| Manual human resolution | Alert marked resolved immediately | Resolved immediately | Pass |
| Two-minute reopen grace | New alert only after ≥2 min | Behaved as configured | Pass |

## 4.5 Results assessment and measurement limits

The following summary table compares the principal scenarios against their recorded status. The subsequent analysis examines how far those observations support broader conclusions.

### *4.5.1 Outcomes of the principal scenarios*

*Table 4.5. Consolidated results for the primary test scenarios*

| Test code | Scenario | Result | Observation |
|---|---|---|---|
| TC01 | Device offline | Pass | Detected after three ping cycles (~45 s) |
| TC02 | Interface down | Pass | Detected through SNMP ifOperStatus |
| TC03 | CPU >85% | Pass | Threshold alert + AI anomaly |
| TC04 | RAM >80% | Pass | AI anomaly; hard threshold not exceeded |
| TC05 | Disk >85% | Pass | Threshold alert |

| Test code | Scenario | Result | Observation |
|---|---|---|---|
| TC06 | Port scan | Pass | Suricata produced relevant event category |
| TC07 | SQL injection | Pass | Suricata recorded an associated signature |
| TC08 | Directory traversal | Pass | Suricata recorded an associated signature |
| TC09 | SSH brute force | Pass | Suricata critical alert |
| TC10 | SYN-flood-like traffic | Pass | Observed at WAN interface |
| TC11 | Traffic anomaly | Pass | Isolation Forest score −0.71 |
| TC12 | Telegram notification | Pass | Recipient routing by role confirmed |
| TC13 | Automatic resolution | Pass | Resolved after 300-second grace period |

### *4.5.2 Detection latency and API performance*

API response times were measured by repeatedly calling the main endpoints 100 times and calculating the arithmetic mean.

/api/devices, returning device snapshots: mean 85 ms with a cache hit and 650 ms when the 12-second cache had expired.

/api/soc/events, returning the 100 most recent events: mean 45 ms.

/api/soc/stats, returning 24-hour security statistics: mean 120 ms.

Resource use: During ordinary operation, NetMon SOC reportedly consumed about 150–200 MB of RAM and less than five percent of host CPU. PostgreSQL with roughly 10,000 soc_events records used about another 100 MB for its buffer pool.

*Table 4.6. Detection delays observed by alert category*

| Alert category | Polling interval | Observed detection time | Explanation |
|---|---|---|---|
| Device offline | 15 s (ping) | 45–60 seconds | Three failures × 15 s + processing |
| Interface down | 15 s (SNMP) | 15–30 seconds | One or two scrape cycles |

| Alert category | Polling interval | Observed detection time | Explanation |
|---|---|---|---|
| CPU/RAM threshold | 15 s | 15–30 seconds | After threshold crossing |
| Suricata alert | 10 s (SSH retrieval) | 10–20 seconds | One or two retrieval cycles |
| AI anomaly | 15 s | 15–30 seconds | Aligned with metrics evaluation |

Detection delay includes the time of the event, the next sample or log-read opportunity, rule evaluation, record creation, and dashboard or Telegram update. Values in the table reflect individual test observations, not a latency distribution such as P95 or P99 and not a measurement under sustained load.

A 12-second cache TTL can make a repeated request substantially faster than a request that must fetch fresh samples from Prometheus. Cache-hit and cache-miss measurements must therefore be reported with their context.

### *4.5.3 False alerts and confidence in conclusions*

A reliable quantitative evaluation needs explicit definitions of true events, false alarms, labeled examples, and a procedure for separating training from testing. The current evidence verifies that the selected laboratory inputs traveled through the collection and notification chain.

Suricata also generated some alerts during legitimate SSH and HTTP administration. The source report recorded an approximate 8–12% false-alert observation, but it did not provide a complete sample count, labeling process, or confusion matrix. Accordingly, this figure should be read as a preliminary indication that rules need tuning for local traffic, not as a statistically validated IDS performance metric.

Isolation Forest flagged the artificially induced CPU, memory, and traffic spikes that were exercised, confirming the training–inference–alert workflow for those conditions. However, contamination=0.05 is an estimator setting rather than a measured false-positive probability.

# CHAPTER 5. OVERALL ASSESSMENT AND FUTURE WORK

## 5.1 Key outcomes and project value

### *5.1.1 Implemented and validated functions*

The system integrates the core functions required for a monitoring laboratory: SNMP and Node Exporter metrics collected by Prometheus; Suricata events from several network zones; threshold alerts with a defined life cycle; Isolation Forest analysis of selected metrics; and a Flask interface connected to Telegram and an AI Agent. The documented scenarios show that the principal processing paths operate in the tested environment.

The six-device PNETLab topology supports repeatable tests involving link loss, resource increases, and controlled security traffic. Chapter 4 records that all 13 principal scenarios met their expected outcomes within the lab. This does not demonstrate accuracy for attack types or operating conditions that were not tested. Detection times vary by data source, polling schedule, and trigger rule; the recorded values are not enterprise service-level objectives.

The layered architecture and Repository Pattern centralize data queries, while device inventories and threshold settings are organized for maintainability. Practical scalability nevertheless remains an open question. Adding more devices, increasing API concurrency, extending data retention, and ingesting many simultaneous events may affect latency and storage costs.

### *5.1.2 Lessons from integrated monitoring design*

A central lesson is to test the entire information path, not just the final screen. A device exposes a value; an exporter or IDS captures it; a storage component records it; detection logic interprets it; and the dashboard or Telegram communicates the result. Each link may fail independently. A convincing monitoring demonstration therefore checks the source metric or EVE JSON event, its persisted representation, and the operator-facing state against a common timeline.

The project also highlights the distinction between an observation and a conclusion. Traffic on an interface does not establish that an application service works; a Suricata signature match does not prove successful exploitation; and an Isolation Forest outlier does not explain the intention behind the workload.

## 5.2 Technical and operational limitations

Project assessment must distinguish successful laboratory operation from readiness for an operational deployment. The six-device topology is appropriate for testing architectural principles but cannot reproduce the diversity, sustained traffic volume, geographic distribution, and compliance obligations of a production enterprise network.

### *5.2.1 Laboratory environment and machine-learning data*

All experiments were performed within PNETLab; the system has not undergone prolonged validation across independent physical hosts or a production network. Observed response time, CPU and memory use, and stability depend partly on the host resources and the particular lab workload.

The approximately seven-day machine-learning baseline does not fully represent weekly, monthly, seasonal, or exceptional activity patterns. Normal workload changes can create outliers even when no attack occurs, so the number of misleading alerts may increase as the environment evolves. A stronger evaluation needs longer historical series, carefully labeled controlled events, and metric-specific assessment before model settings or severity thresholds are generalized.

### *5.2.2 Application security and long-term retention*

The web application does not yet provide complete user authentication and authorization. Although service keys are stored in environment variables, deployment on an accessible network would require TLS, session management, least-privilege roles, input validation, and access logs.

The AI Agent provides explanations and investigation suggestions but does not automatically execute configuration changes. This is a deliberate safety boundary: firewall-rule modifications, device commands, or remediation actions should be

approved through an explicit operational change process and verified against device state before being applied.

Security events and operational alerts accumulate over time, but the lab version has not validated a comprehensive long-term retention, rotation, archival, and disaster-recovery strategy. Limiting the number of records returned by an API is not equivalent to controlling database growth.

## 5.3 Development roadmap

A responsible roadmap should prioritize safe and reproducible operation before adding advanced automation. The proposed sequence is to harden access control and TLS, establish reliable storage and backup, build an independently labeled evaluation dataset, and only then consider semi-automated response capabilities.

### *5.3.1 Broader monitoring, security, and response capabilities*

Additional sources could include NetFlow or sFlow for flow-level traffic observations and application exporters for Nginx and PostgreSQL. The objective would be greater visibility into the network and services, balanced against collection overhead, metric-label cardinality, storage growth, and retention cost.

Automated responses such as temporarily blocking an IP address or querying host processes could first be studied in an isolated environment with human approval, an allowlist, time limits, audit logs, and rollback. The language model should not receive unrestricted device-control privileges on the basis of a single alert, because alert evidence can be incomplete or misleading.

Before broader use, the interface and APIs require login authentication, role-based authorization, HTTPS, and rate limiting. Secure token and SSH-key management, careful input handling, and access auditing should accompany those changes.

### *5.3.2 Better anomaly detection and operator experience*

Once sufficient historical and labeled data are available, Isolation Forest can be compared with approaches that explicitly represent periodic or time-dependent behavior.

Evaluation should report precision, recall, false-positive rate, and detection delay separately for relevant anomaly categories.

The interface could support periodic operational reports, more flexible investigation filters, and a mobile-friendly presentation of incidents. Every extension should preserve the link between displayed interpretations and their underlying source records, including data timestamps and processing status.

The recommended implementation order is: normalize data sources; validate access security and retention; construct a labeled benchmark; compare anomaly models; and test supervised or approval-based response workflows.

# CONCLUSION

NetMon SOC combines device-state information from SNMP and Node Exporter, Suricata security events, service-health observations, and Isolation Forest outlier scores in a single operational workflow. Flask and PostgreSQL organize application records and views; Telegram notifies responsible operators; and the AI Agent helps interpret current context. The result is a useful laboratory platform for practicing fault isolation and comparing signals from network infrastructure and security monitoring.

According to the documented tests, the thirteen principal scenarios met their expected outcomes in the PNETLab model. This statement does not mean that the system has been certified on production networks, has a verified general detection-accuracy rate, or can sustain prolonged enterprise-scale workloads. The project's practical value lies in reproducible implementation steps, evidence-driven validation, and clearly stated boundaries that guide future improvements in access security, labeled-data evaluation, and operational control.

# REFERENCES

[1] Bejtlich, R. (2004). The Tao of Network Security Monitoring: Beyond Intrusion Detection. Addison-Wesley Professional, Boston, MA. Available at: https://www.pearson.com/en-us/subject-catalog/p/tao-of-network-security-monitoring-the-beyond-intrusion-detection/P200000009342

[2] Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation Forest. 2008 Eighth IEEE International Conference on Data Mining, 413–422. IEEE. Available at: https://ieeexplore.ieee.org/document/4781136

[3] Prometheus Authors. (2023). Prometheus Documentation — Querying and Storage. Prometheus.io. Available at: https://prometheus.io/docs/introduction/overview/

[4] Open Information Security Foundation (OISF). (2023). Suricata User Guide. Available at: https://docs.suricata.io/en/latest/

[5] Nguyễn Gia Thọ et al. (2021). Network Security and Information Protection: Course Textbook [in Vietnamese]. Vietnam National University Press, Hanoi. Available at: https://repository.vnu.edu.vn/handle/VNU_123/144256

[6] Vietnam Computer Emergency Response Center (VNCERT/CC). (2023). Guidelines for Establishing a Security Operations Center for Organizations [in Vietnamese]. Authority of Information Security, Ministry of Information and Communications. Available at: https://ais.gov.vn/huong-dan-xay-dung-soc